

# UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

DIPARTIMENTO DI INFORMATICA, SISTEMISTICA E  
COMUNICAZIONE

CORSO DI LAUREA MAGISTRALE IN INFORMATICA



## Architettura Dati

Valutazione dell'Impatto del Rumore sulla Predizione dei  
Modelli di Machine Learning

**Autore:**

Alen Bicanic Matr. 945230

Samuele Perotti Matr. 899817

Anno Accademico 2025-2026

# Indice

|          |                                                                                   |           |
|----------|-----------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                                               | <b>3</b>  |
| <b>2</b> | <b>Descrizione del dataset</b>                                                    | <b>4</b>  |
| <b>3</b> | <b>Preprocessing dei dati</b>                                                     | <b>6</b>  |
| 3.1      | Rimozione di colonne non informative . . . . .                                    | 6         |
| 3.2      | Conversione del target in problema binario . . . . .                              | 6         |
| 3.3      | Gestione delle variabili booleane . . . . .                                       | 6         |
| 3.4      | One-hot encoding delle variabili categoriche . . . . .                            | 6         |
| 3.5      | Gestione dei valori mancanti residui . . . . .                                    | 6         |
| 3.6      | Scaling delle feature . . . . .                                                   | 7         |
| <b>4</b> | <b>Analisi esplorativa (EDA)</b>                                                  | <b>8</b>  |
| 4.1      | Distribuzione del target binario . . . . .                                        | 8         |
| 4.2      | Matrice di correlazione . . . . .                                                 | 9         |
| 4.3      | Distribuzione delle variabili numeriche . . . . .                                 | 10        |
| <b>5</b> | <b>Suddivisione train/test</b>                                                    | <b>11</b> |
| <b>6</b> | <b>Modelli di baseline</b>                                                        | <b>11</b> |
| 6.1      | Decision Tree . . . . .                                                           | 11        |
| 6.1.1    | Decision Tree - Pruned . . . . .                                                  | 12        |
| 6.2      | Neural Network (Multi-Layer Perceptron) . . . . .                                 | 12        |
| 6.3      | Support Vector Machine (SVM) . . . . .                                            | 15        |
| 6.4      | Confronto tra i modelli di baseline . . . . .                                     | 15        |
| <b>7</b> | <b>Introduzione del rumore nel dataset</b>                                        | <b>17</b> |
| 7.1      | Valori mancanti (NaN) . . . . .                                                   | 17        |
| 7.2      | Valori duplicati . . . . .                                                        | 17        |
| 7.3      | Dati "sporchi" . . . . .                                                          | 17        |
| <b>8</b> | <b>Impatto del rumore sulle prestazioni dei modelli</b>                           | <b>19</b> |
| 8.1      | Valori mancanti . . . . .                                                         | 19        |
| 8.1.1    | Cross-Validation . . . . .                                                        | 23        |
| 8.2      | Valori duplicati . . . . .                                                        | 24        |
| 8.2.1    | Cross-Validation . . . . .                                                        | 27        |
| 8.3      | Dati sporchi . . . . .                                                            | 28        |
| 8.3.1    | Cross Validation . . . . .                                                        | 31        |
| <b>9</b> | <b>Tecniche di preprocessing e regolarizzazione per la mitigazione del rumore</b> | <b>33</b> |
| 9.1      | Mitigazione dei valori mancanti . . . . .                                         | 33        |
| 9.1.1    | Imputazione con KNN . . . . .                                                     | 33        |
| 9.1.2    | Imputazione Iterativa . . . . .                                                   | 37        |
| 9.2      | Mitigazione dei valori duplicati . . . . .                                        | 42        |
| 9.2.1    | Rimozione dei duplicati . . . . .                                                 | 43        |
| 9.2.2    | Aggregazione intelligente . . . . .                                               | 44        |
| 9.2.3    | Confronto tra le strategie e conclusioni . . . . .                                | 46        |

|                                             |           |
|---------------------------------------------|-----------|
| 9.3 Mitigazione dei dati sporchi . . . . .  | 47        |
| <b>10 Confronto finale tra le strategie</b> | <b>51</b> |
| <b>11 Conclusioni</b>                       | <b>52</b> |

# 1 Introduzione

L'obiettivo del progetto è comprendere in che modo la presenza di rumore nei dati influenzi le prestazioni dei modelli di Machine Learning e individuare le strategie più efficaci per limitarne l'impatto. Il lavoro si è articolato nelle seguenti fasi, in linea con la proposta progettuale concordata:

- scelta e analisi di un dataset adatto al problema di classificazione;
- preparazione dei dati e creazione del training set;
- introduzione di rumore nel dataset a diversi livelli di intensità;
- addestramento di più modelli sul dataset modificato e analisi delle prestazioni al variare del rumore;
- applicazione di tecniche di preprocessing e regolarizzazione (normalizzazione/scaling, PCA, cross-validation, early stopping, dropout, imputazione avanzata, gestione dei duplicati) per valutarne l'efficacia nel contenere il degrado delle prestazioni;
- confronto tra i risultati ottenuti sui dataset rumorosi e quelli ottenuti sul dataset originale;
- valutazione delle performance mediante le metriche di accuracy, precision e recall.

Il dominio applicativo scelto è quello medico/di supporto alla diagnosi: si è utilizzato l'*UCI Heart Disease Dataset*, con l'obiettivo di prevedere la presenza o assenza di una malattia cardiaca a partire da variabili cliniche (classificazione binaria).

## 2 Descrizione del dataset

Il dataset utilizzato è lo **UCI Heart Disease Dataset**, reso disponibile su GitHub<sup>1</sup> e caricato in questo progetto da un repository GitHub del corso. Il dataset è composto da **920 istanze** e, nella sua forma originale, da **16 colonne**, tra cui variabili demografiche (età, sesso), cliniche (pressione a riposo, colesterolo, frequenza cardiaca massima, angina da sforzo, ecc.) e la variabile target **num**.

La variabile target originale è **multiclasse** (valori da 0 a 4, dove 0 indica assenza di malattia e i valori da 1 a 4 indicano livelli crescenti di gravità). La distribuzione originale delle classi è riportata in Tabella 1.

| Classe (num) | Numero di istanze |
|--------------|-------------------|
| 0 (assenza)  | 411               |
| 1            | 265               |
| 2            | 109               |
| 3            | 107               |
| 4            | 28                |

Tabella 1: Distribuzione originale della variabile target multiclasse.

Diverse variabili presentano valori mancanti in misura significativa (ad esempio la colonna **ca**, presente solo in 309 istanze su 920), aspetto che ha reso necessaria un'attenta fase di preprocessing, descritta nella sezione seguente.

- **id**: identificatore univoco del paziente.
- **age**: età del paziente espressa in anni.
- **origin (dataset)**: luogo di provenienza dello studio clinico.
- **sex**: sesso del paziente (Male/Female).
- **cp (chest pain type)**: tipologia di dolore toracico, con le seguenti modalità:
  - typical angina
  - atypical angina
  - non-anginal pain
  - asymptomatic
- **trestbps**: pressione arteriosa a riposo (in mm Hg) misurata al momento del ricovero.
- **chol**: livello di colesterolo (mg/dl).
- **fbs (fasting blood sugar)**: indica se la glicemia a digiuno è superiore a 120 mg/dl (True/False).

---

<sup>1</sup>[https://raw.githubusercontent.com/Bicanic-Alen/Progetto\\_Archittura\\_Dati\\_2026/refs/heads/master/heart\\_disease\\_uci.csv](https://raw.githubusercontent.com/Bicanic-Alen/Progetto_Archittura_Dati_2026/refs/heads/master/heart_disease_uci.csv)

- **restecg**: risultati dell'elettrocardiogramma a riposo:
  - normal
  - ST-T wave abnormality
  - left ventricular hypertrophy
- **thalach (maximum heart rate achieved)**: frequenza cardiaca massima raggiunta durante il test.
- **exang**: presenza di angina indotta dall'esercizio fisico (True/False).
- **oldpeak**: depressione del tratto ST indotta dall'esercizio rispetto alla condizione di riposo.
- **slope**: pendenza del segmento ST al picco dell'esercizio.
- **ca**: numero di vasi principali (0–3) colorati tramite fluoroscopia.
- **thal**: risultato del test al tallio:
  - normal
  - fixed defect
  - reversible defect
- **num**: variabile target originale, che indica la presenza e la severità della malattia cardiaca.

### 3 Preprocessing dei dati

Prima dell'addestramento dei modelli, il dataset è stato sottoposto alle seguenti operazioni di pulizia e trasformazione.

#### 3.1 Rimozione di colonne non informative

Le colonne `id` (identificativo univoco della riga) e `dataset` (indicante solo la provenienza geografica del campione) sono state rimosse in quanto non contribuiscono alla predizione.

#### 3.2 Conversione del target in problema binario

Per semplicità e coerenza clinica, il target multiclasse `num` è stato trasformato in una variabile binaria `target`:

- 0 = assenza di malattia;
- 1 = presenza di malattia (aggregazione delle classi originarie 1–4).

#### 3.3 Gestione delle variabili booleane

Le variabili `fbs` (glicemia a digiuno) ed `exang` (angina indotta da sforzo) sono state convertite in formato numerico 0/1, imputando i valori mancanti con `False` prima della conversione.

#### 3.4 One-hot encoding delle variabili categoriche

Le variabili categoriche `sex`, `cp`, `restecg`, `slope` e `thal` sono state trasformate tramite *one-hot encoding* (con rimozione della prima categoria per evitare collinearità, `drop_first=True`).

#### 3.5 Gestione dei valori mancanti residui

Dopo l'encoding, le colonne numeriche che presentavano ancora valori mancanti (`trestbps`, `chol`, `thalch`, `oldpeak`, `ca`) sono state imputate con la **mediana** della rispettiva colonna. Questa scelta è stata preferita alla media in quanto più robusta rispetto agli outlier e permette di non perdere istanze, aspetto particolarmente rilevante data la dimensione limitata del dataset. Il conteggio dei valori mancanti per colonna, riscontrato dopo l'encoding, è riportato in Tabella 2.

| Colonna               | Valori mancanti |
|-----------------------|-----------------|
| <code>trestbps</code> | 59              |
| <code>chol</code>     | 30              |
| <code>thalch</code>   | 55              |
| <code>oldpeak</code>  | 62              |
| <code>ca</code>       | 611             |

Tabella 2: Valori mancanti per colonna numerica prima dell'imputazione.

### 3.6 Scaling delle feature

Le variabili numeriche sono state normalizzate/scalate (StandardScaler) per garantire che feature con scale di grandezza diverse (es. colesterolo vs età) non sbilanciassero il peso durante l'addestramento, passaggio essenziale soprattutto per la Rete Neurale.



## 4 Analisi esplorativa (EDA)

Dopo il preprocessing è stata condotta un'analisi esplorativa per comprendere la struttura del dataset ripulito.

### 4.1 Distribuzione del target binario

Il grafico a barre della distribuzione del target binario mostra un discreto bilanciamento tra le due classi (assenza/presenza di malattia), condizione favorevole per l'addestramento dei modelli di classificazione.

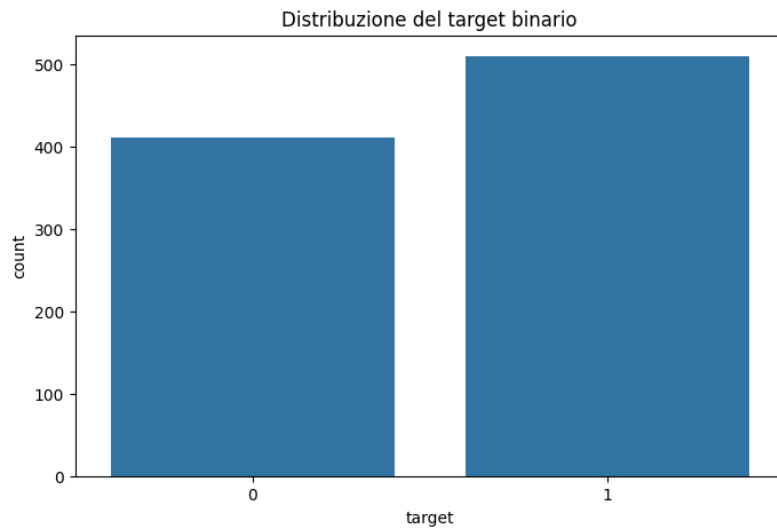


Figura 1: Distribuzione della variabile target binaria.

## 4.2 Matrice di correlazione

La matrice di correlazione tra tutte le variabili numeriche del dataset permette di individuare eventuali relazioni lineari forti tra le feature e con il target.

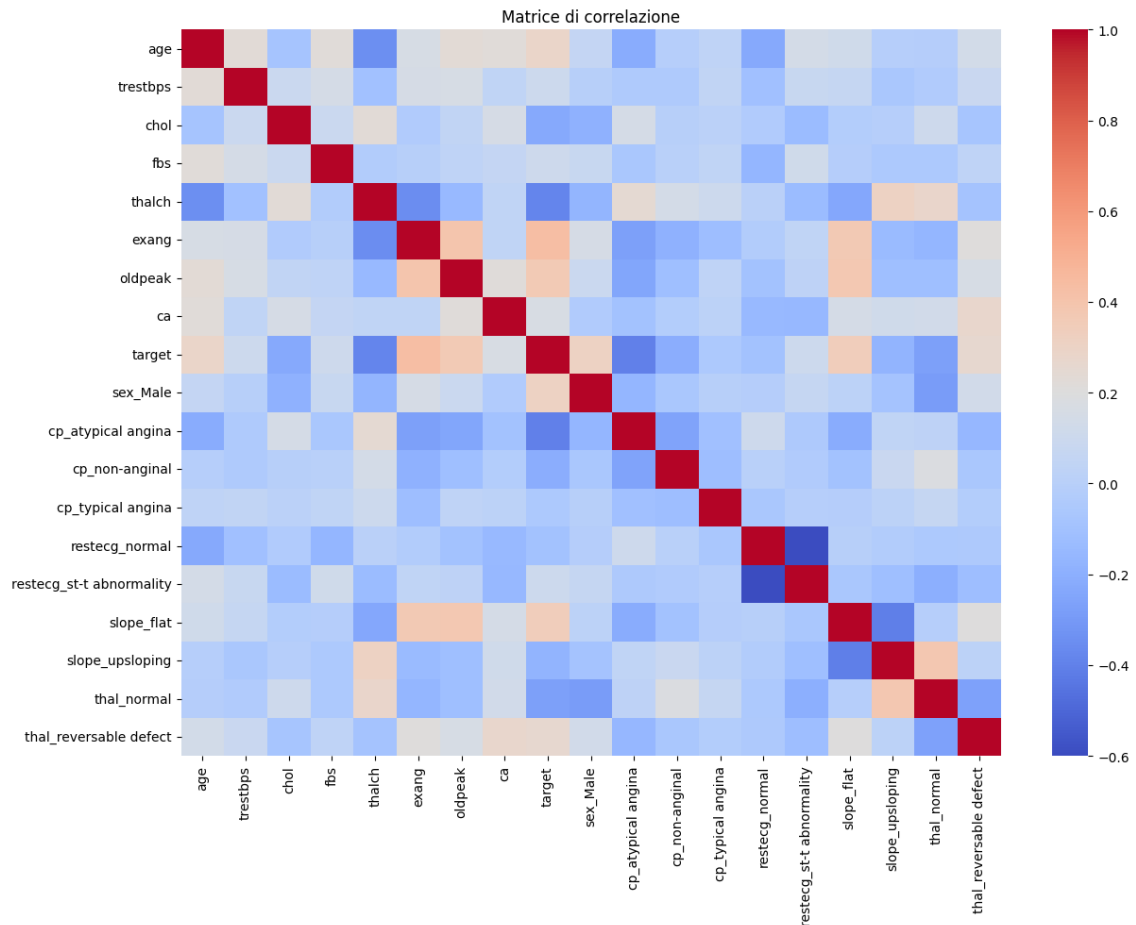


Figura 2: Matrice di correlazione tra le variabili del dataset

Dall'analisi esplorativa dei dati riportata nel notebook, è emerso attraverso la matrice di correlazione che variabili come il tipo di dolore toracico (cp) e la frequenza cardiaca massima (thalach) mostrano una correlazione positiva con la presenza della malattia. Al contrario, variabili come oldpeak mostrano una correlazione negativa. Inoltre, i grafici di distribuzione hanno evidenziato come l'età sia un fattore di rischio distribuito prevalentemente nella fascia over-50.

### 4.3 Distribuzione delle variabili numeriche

Gli istogrammi delle variabili numeriche permettono di valutarne la forma distributiva e la presenza di eventuali asimmetrie o outlier.

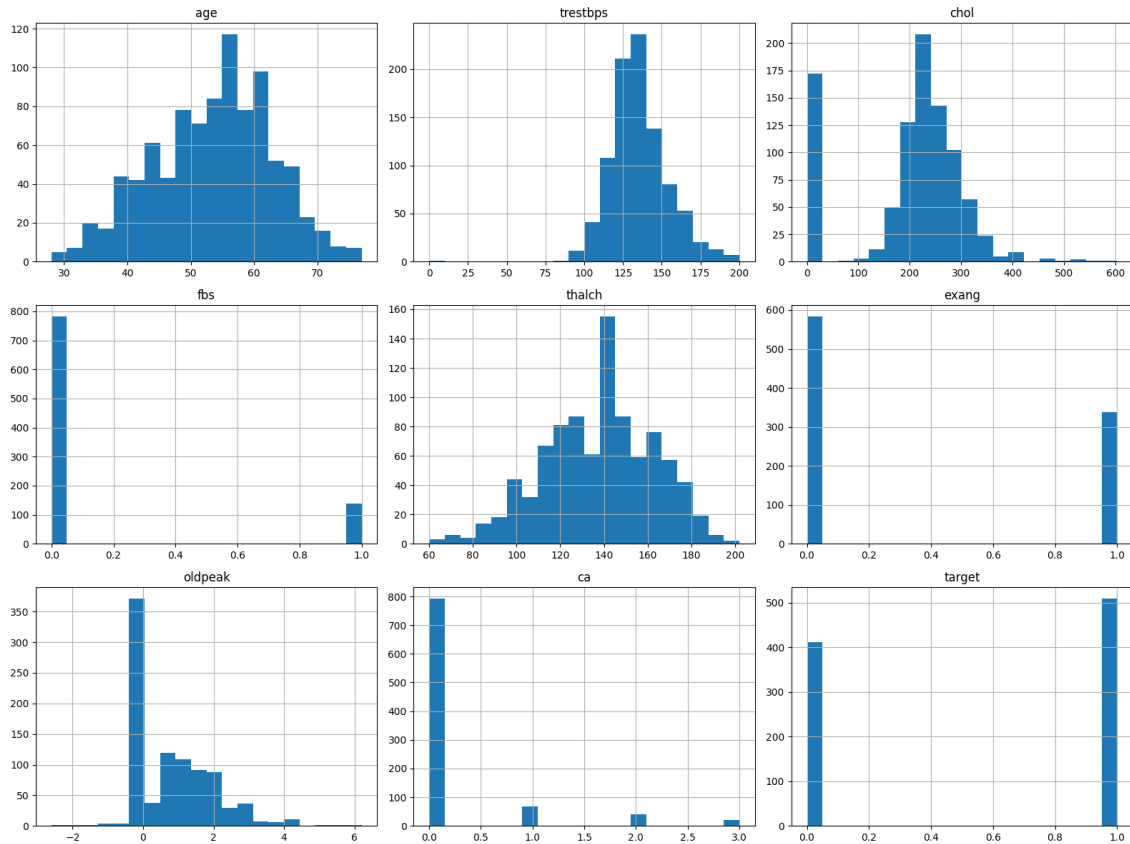


Figura 3: Istogrammi delle variabili numeriche

Gli istogrammi delle variabili numeriche mostrano distribuzioni non gaussiane e la presenza di asimmetrie, rafforzando la scelta della mediana come strategia di imputazione. I risultati dell'analisi esplorativa hanno fornito indicazioni fondamentali per le successive scelte di preprocessing, in particolare per la gestione dei valori mancanti, la codifica delle variabili categoriche e l'applicazione dello scaling delle feature.

## 5 Suddivisione train/test

Il dataset è stato suddiviso in training set e test set con proporzione 80/20 (`test_size=0.2`), utilizzando la stratificazione sulla variabile target (`stratify=y`) per preservare le proporzioni delle classi in entrambi i sottoinsiemi, e fissando `random_state=42` per la riproducibilità dei risultati.

## 6 Modelli di baseline

Sono stati addestrati e confrontati tre modelli di classificazione supervisionata sul dataset originale (privo di rumore aggiuntivo).

### 6.1 Decision Tree

È stato inizialmente addestrato un Decision Tree con criterio *entropy* senza vincoli di profondità, ottenendo i seguenti risultati sul test set:

| Metrica   | Classe 0 | Classe 1 | Media pesata |
|-----------|----------|----------|--------------|
| Precision | 0.73     | 0.77     | 0.75         |
| Recall    | 0.71     | 0.79     | 0.75         |
| F1-score  | 0.72     | 0.78     | 0.75         |

Tabella 3: Decision Tree (non potato) – Accuracy sul test set: 0.7554

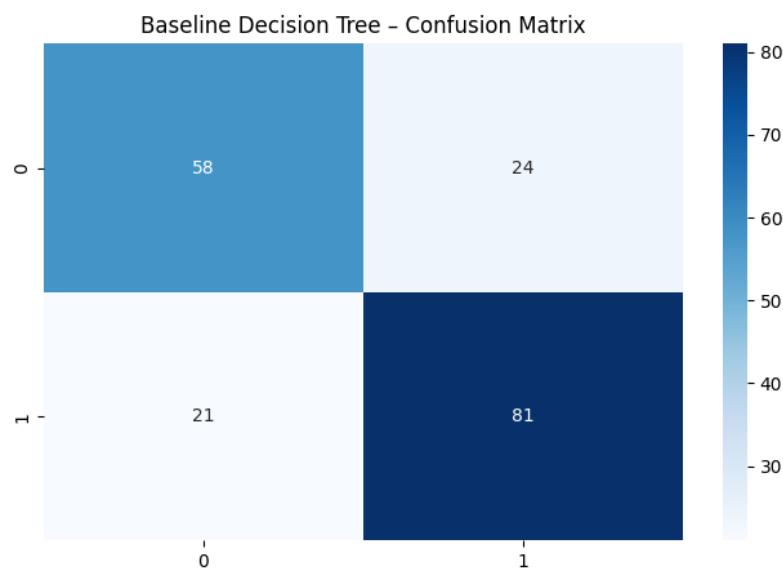


Figura 4: Matrice di confusione – Decision Tree (baseline)

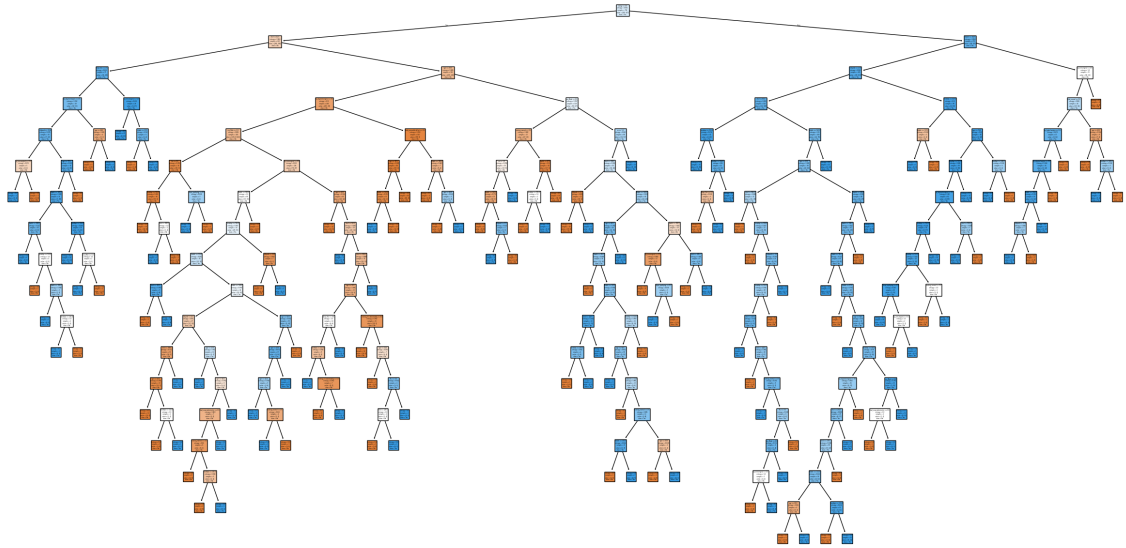


Figura 5: Albero di decisione non potato

### 6.1.1 Decision Tree - Pruned

L'albero non potato tende a sovradattare (*overfitting*) i dati di training. Per contenere questo effetto è stato introdotto un vincolo sulla profondità massima (`max_depth=5`), ottenendo un modello più semplice, più interpretabile e con prestazioni migliori sul test set:

| Metrica   | Classe 0 | Classe 1 | Media pesata |
|-----------|----------|----------|--------------|
| Precision | 0.80     | 0.81     | 0.80         |
| Recall    | 0.74     | 0.85     | 0.80         |
| F1-score  | 0.77     | 0.83     | 0.80         |

Tabella 4: Decision Tree potato (`max_depth=5`) – Accuracy sul test set: 0.8043

La validazione tramite 5-fold cross-validation sul Decision Tree potato ha restituito un'accuracy media pari a **0.725**, coerente con quanto osservato sul singolo test set.

## 6.2 Neural Network (Multi-Layer Perceptron)

È stata definita una rete neurale feed-forward per la classificazione binaria, con la seguente architettura:

- Strato di input + primo hidden layer: 16 neuroni, attivazione sigmoide;
- Secondo hidden layer: 8 neuroni, attivazione sigmoide;
- Strato di output: 1 neurone, attivazione sigmoide (adatta alla classificazione binaria).

Il modello è stato addestrato per 100 epoche con `batch_size=16` sui dati precedentemente standardizzati (`StandardScaler`), passaggio necessario poiché le reti neurali sono sensibili alla scala delle feature in input.

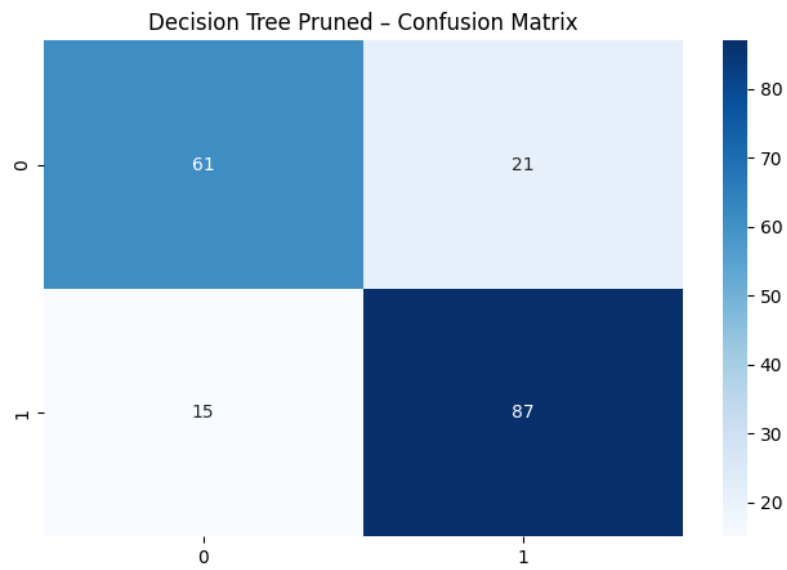


Figura 6: Matrice di confusione – Decision Tree Potato

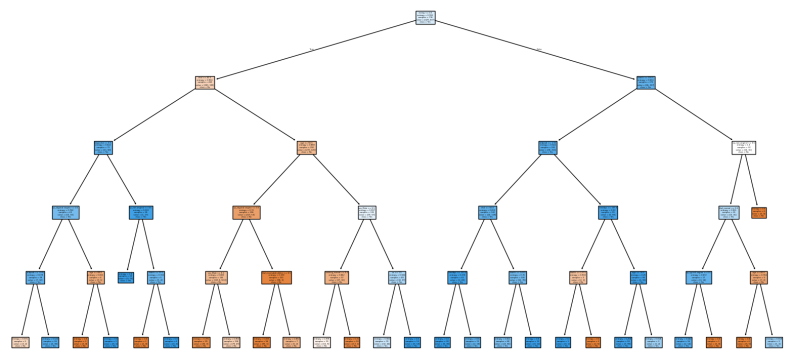


Figura 7: Matrice di confusione – Decision Tree Potato

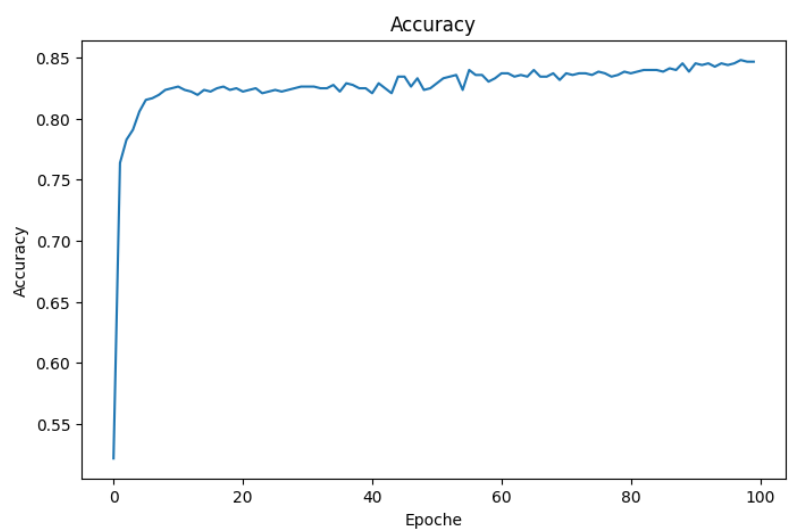


Figura 8: Andamento dell'accuracy di training (Neural Network)

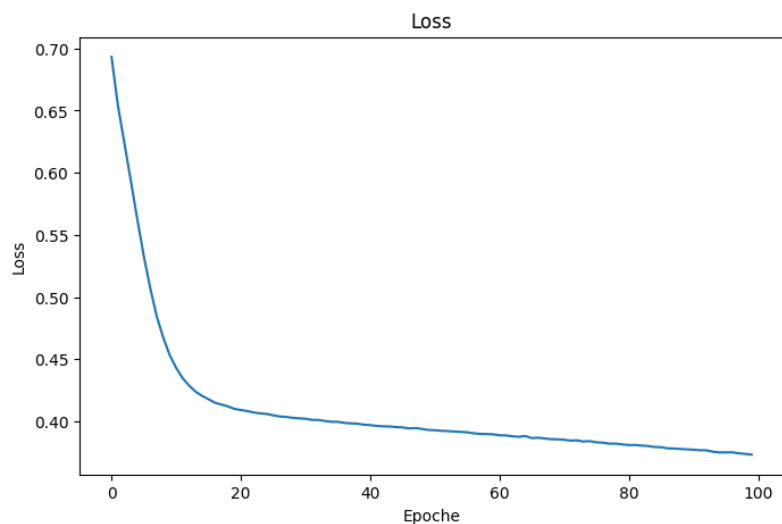


Figura 9: Andamento della loss di training (Neural Network)

Sul test set, il modello ha ottenuto una **Test Accuracy di 0.8315** (Test Loss = 0.3813), con le seguenti metriche di classificazione:

| Metrica   | Classe 0 | Classe 1 | Media pesata |
|-----------|----------|----------|--------------|
| Precision | 0.83     | 0.83     | 0.83         |
| Recall    | 0.78     | 0.87     | 0.83         |
| F1-score  | 0.81     | 0.85     | 0.83         |

Tabella 5: Neural Network – Accuracy sul test set: 0.8315

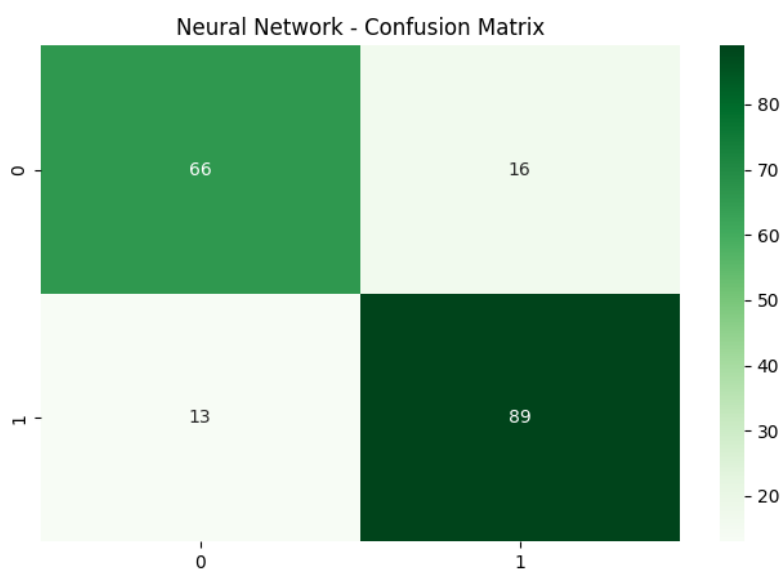


Figura 10: Matrice di confusione - Neural Network

### 6.3 Support Vector Machine (SVM)

Per il terzo modello è stata scelta una SVM con le seguenti motivazioni:

- **Kernel RBF (Radial Basis Function)**: a differenza di un kernel lineare, il kernel RBF gestisce relazioni non lineari tra le feature; dato che le variabili cliniche (pressione, colesterolo, età, ecc.) spesso interagiscono in modo complesso, un confine di decisione non lineare tende a offrire prestazioni migliori;
- **Parametro di regolarizzazione  $C = 1.0$** : controlla il compromesso (*trade-off*) tra l'ampiezza del margine di separazione e la corretta classificazione dei punti di training.

Il modello, addestrato sui dati standardizzati, ha ottenuto un' **Accuracy di 0.8424** sul test set:

| Metrica   | Classe 0 | Classe 1 | Media pesata |
|-----------|----------|----------|--------------|
| Precision | 0.90     | 0.81     | 0.85         |
| Recall    | 0.73     | 0.93     | 0.84         |
| F1-score  | 0.81     | 0.87     | 0.84         |

Tabella 6: SVM (kernel RBF,  $C=1.0$ ) – Accuracy sul test set: 0.8424

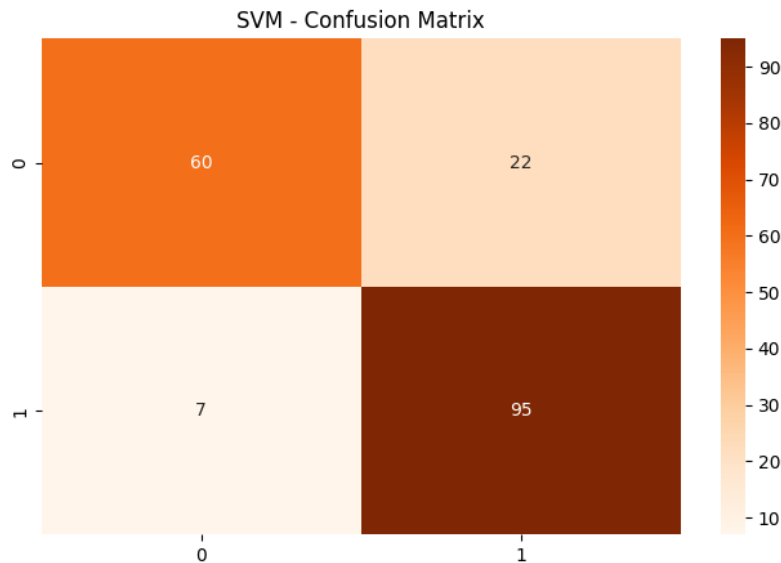


Figura 11: Matrice di confusione - SVM

### 6.4 Confronto tra i modelli di baseline

La Tabella 7 riassume le prestazioni dei quattro modelli addestrati (Decision Tree non potato, Decision Tree potato, Neural Network, SVM) sul dataset originale, privo di rumore.



| Modello                  | Accuracy | Precision | Recall | F1-score |
|--------------------------|----------|-----------|--------|----------|
| Decision Tree (baseline) | 0.755    | 0.75      | 0.75   | 0.75     |
| Decision Tree (potato)   | 0.804    | 0.80      | 0.80   | 0.80     |
| Neural Network           | 0.832    | 0.83      | 0.83   | 0.83     |
| SVM                      | 0.842    | 0.85      | 0.84   | 0.84     |

Tabella 7: Confronto delle metriche sul test set tra i modelli di baseline (valori pesati/medi).

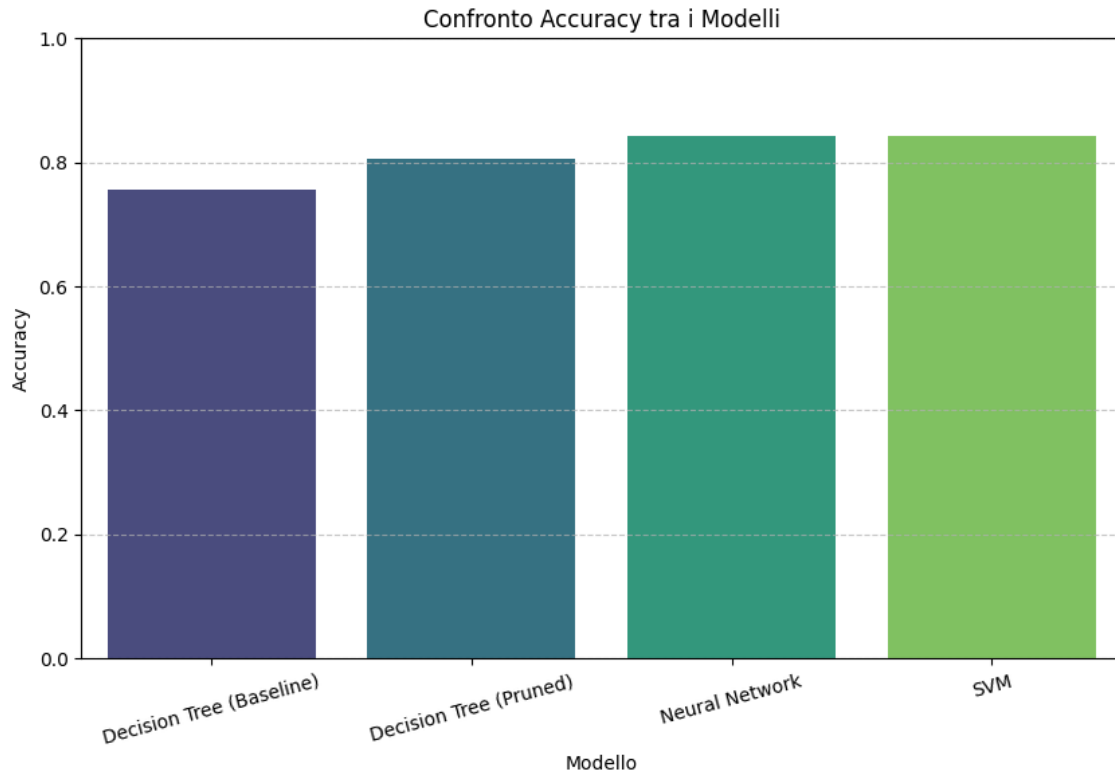


Figura 12: Confronto dell'Accuracy tra i modelli di baseline

La SVM con kernel RBF risulta il modello con le prestazioni migliori sul test set, seguita dalla rete neurale; il Decision Tree potato mostra un netto miglioramento rispetto alla versione non potata, a conferma dell'efficacia del pruning nel ridurre l'overfitting.

## 7 Introduzione del rumore nel dataset

Per valutare la robustezza dei modelli, è stato introdotto rumore artificiale nel training set (il test set rimane sempre pulito, per garantire un confronto equo tra le diverse condizioni), a tre livelli di intensità crescente:

| Livello | Percentuale di rumore |
|---------|-----------------------|
| Low     | 15%                   |
| Medium  | 40%                   |
| High    | 55%                   |

Tabella 8: Livelli di rumore utilizzati negli esperimenti.

Sono state simulate tre diverse tipologie di rumore, ciascuna rappresentativa di problematiche comuni nei dataset reali.

### 7.1 Valori mancanti (NaN)

È stata implementata una funzione che introduce valori NaN in modo casuale nel training set, generando per ogni cella un numero casuale e sostituendo il valore originale con NaN se tale numero è inferiore alla soglia di rumore desiderata. Il rumore è stato applicato *esclusivamente* su `X_train`, in modo da poter confrontare in modo equo le prestazioni sul medesimo test set pulito. La percentuale effettiva di valori mancanti introdotti è risultata coerente con quella target (15.2%, 40.4%, 55.7% rispettivamente).

### 7.2 Valori duplicati

È stata implementata una funzione che duplica una percentuale di righe del training set, scelte casualmente (con reinserimento), simulando errori di raccolta/unione dati che generano osservazioni ripetute. Le dimensioni del training set risultante sono riportate in Tabella 9.

| Livello      | Dimensione training set | Dimensione originale |
|--------------|-------------------------|----------------------|
| Low (15%)    | 846                     | 736                  |
| Medium (40%) | 1030                    | 736                  |
| High (55%)   | 1140                    | 736                  |

Tabella 9: Dimensione del training set dopo l'inserimento di duplicati.

### 7.3 Dati "sporchi"

È stata implementata una funzione che introduce rumore realistico differenziato in base al tipo di variabile:

- sulle colonne **continue**, viene aggiunto rumore gaussiano proporzionale alla deviazione standard della colonna stessa, applicato solo a una frazione casuale di righe (in base al livello di rumore);

- sulle colonne **binarie** (incluse quelle ottenute da one-hot encoding), viene effettuato uno *swap* dei valori (0 diventa 1 e viceversa) su una frazione casuale di righe.

Questa tipologia di rumore simula errori di misurazione/inserimento tipici dei dati reali, più insidiosi dei semplici valori mancanti perché non facilmente identificabili.

## 8 Impatto del rumore sulle prestazioni dei modelli

Per ciascuna tipologia di rumore, i tre modelli (Decision Tree, SVM, Neural Network) sono stati riaddestrati sul training set corrotto e valutati sul medesimo test set pulito, applicando un preprocessing di base (imputazione con mediana per i NaN, scaling standard). Le prestazioni sono state inoltre validate tramite 5-fold cross-validation stratificata (`StratifiedKFold`,  $k=5$ ) sul training set corrotto.

### 8.1 Valori mancanti

La prima tipologia di rumore che viene tratta è il rumore dovuto ai valori nulli. Per eseguire i test si è modificato il dataset introducendo a scaglioni i valori nulli con le percentuali indicate nel precedente capitolo. Per ogni scaglione è inclusa la versione baseline 0% (il dataframe sprovvisto da rumore) vengono addestrati i tre algoritmi di Machine Learning.

Dopo di che gli algoritmi vengono addestrati, e vengono prodotte le seguenti informazioni, la matrice di confusione, i valori dell'Accuracy, Precision, Recall e F1-Score e un grafico sul accuracy. Tali informazioni ci permettono di capire il comportamento del algoritmo per ogni specifico caso.

Successivamente eseguiamo anche la cross-validation k-fold, con  $k = 5$  il quale ci permette di distinguere le prestazioni da un caso fortunato o sfortunato, dandoci dei valori più generali del comportamento degli algoritmi, non solo per un specifico caso.

Di seguito si osservano le matrici di confusione:

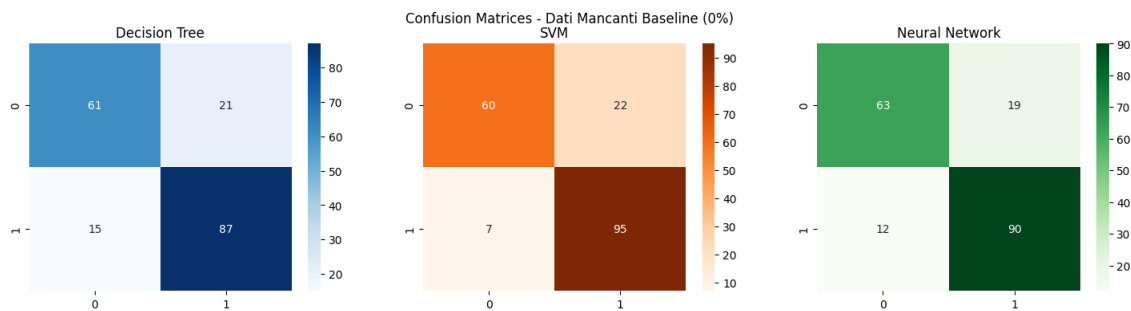


Figura 13: Matrice di confusione della baseline

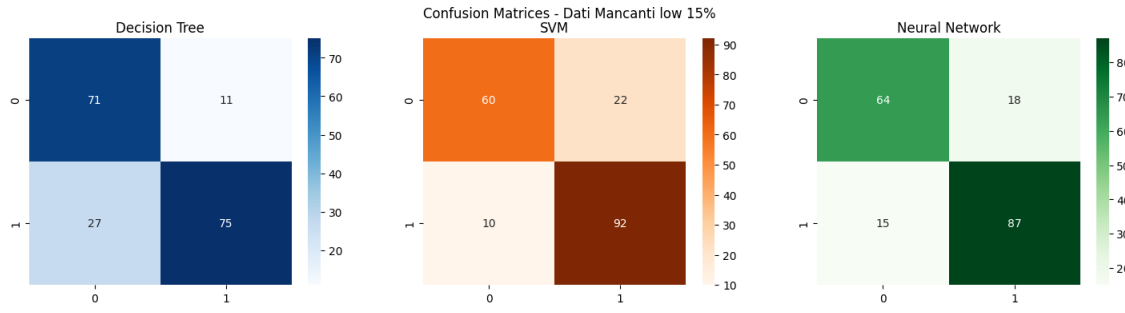


Figura 14: Matrice di confusione con l'introduzione del 15% di valori nulli

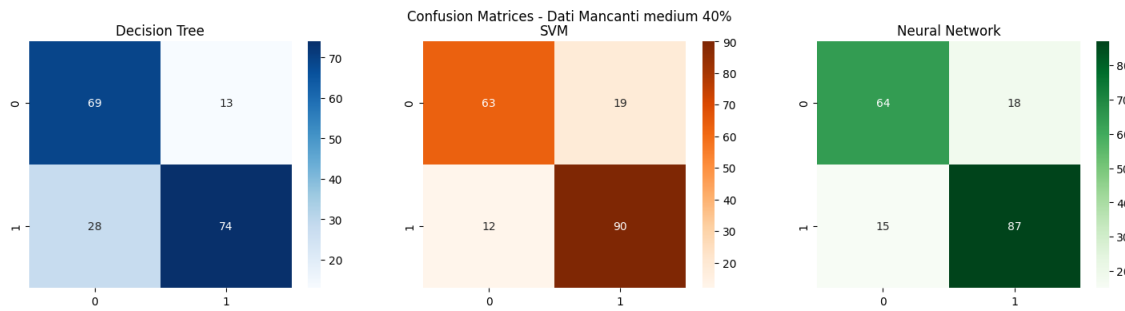


Figura 15: Matrice di confusione con l'introduzione del 40% di valori nulli

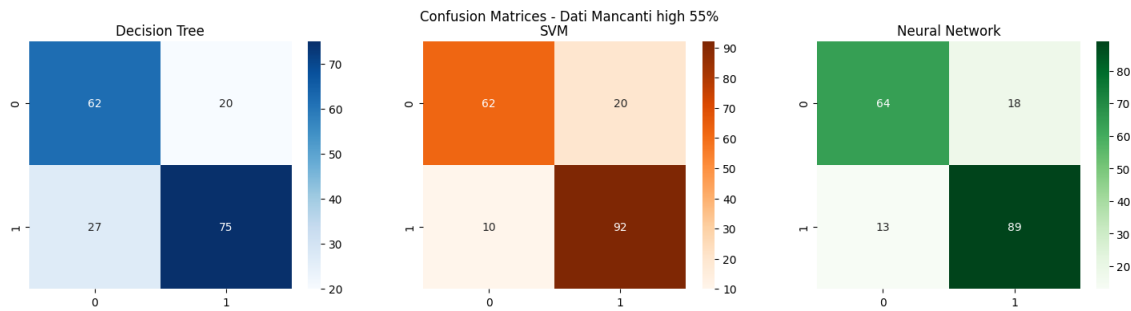


Figura 16: Matrice di confusione con l'introduzione del 55% di valori nulli

Dalle matrici di confusione mostrate nelle Figure 13, 14, 15 e 16, si osservano sulla diagonale principale i valori predetti correttamente (rispettivamente, dall'alto verso il basso, i Veri Negativi e i Veri Positivi), mentre al di fuori di essa (in alto a destra e in basso a sinistra) figurano i valori classificati in modo errato, ovvero i Falsi Positivi e i Falsi Negativi.

La Figura 13 riporta i risultati sul dataset originale (pulito), che rappresenta la baseline di riferimento per valutare il successivo degrado delle performance. In questo scenario si nota una distribuzione ottimale, con un'elevata concentrazione di valori sulla diagonale; la completezza dei dati permette di minimizzare i Falsi Negativi, garantendo un'elevata affidabilità clinica. È già possibile notare come i modelli SVM e Rete Neurale mostrino una precisione superiore rispetto al Decision Tree.

Con l'introduzione progressiva del 15% e del 40% di rumore (Figure 14 e 15), le prestazioni della SVM e della Rete Neurale rimangono pressoché stabili rispetto alla baseline. Al contrario, il Decision Tree inizia a registrare un aumento dei Falsi Negativi. Questo indica che la ricostruzione dei dati tramite imputazione, per quanto efficace, introduce un rumore sufficiente a distorcere i confini decisionali gerarchici tipici di questo modello.

Infine, nello scenario più critico in cui la percentuale di valori nulli supera quella dei dati reali (55%, mostrato in Figura 16), si osserva un netto divario tra le architetture:

- Il Decision Tree mostra il comportamento peggiore, registrando ben 27 Falsi Negativi, un dato che in un contesto clinico è preoccupante. La notevole perdita di informazioni compromette in maniera irreversibile i criteri di split nodale, impedendo all'algoritmo di individuare un numero elevato di pazienti positivi.
- La SVM si dimostra invece l'algoritmo in assoluto più resiliente, limitando i Falsi Negativi a soli 10 casi e mantenendo ben 92 Veri Positivi (un valore vicinissimo ai 95 della baseline). Ciò conferma la capacità del kernel RBF di garantire un iperpiano di separazione estremamente solido.
- La Rete Neurale, infine, si conferma la soluzione più bilanciata: offrendo ottime prestazioni con soli 13 Falsi Negativi, dimostra che un apprendimento basato sull'aggiornamento dei pesi sinaptici e sulle funzioni di attivazione è in grado di tollerare efficacemente il rumore introdotto dall'imputazione semplice.

| Livello         | Modello        | Accuracy | Precision | Recall | F1-Score |
|-----------------|----------------|----------|-----------|--------|----------|
| <i>Baseline</i> | Decision Tree  | 0.804    | 0.806     | 0.853  | 0.829    |
|                 | SVM            | 0.842    | 0.812     | 0.931  | 0.868    |
|                 | Neural Network | 0.832    | 0.826     | 0.882  | 0.853    |
| <i>Low 15%</i>  | Decision Tree  | 0.793    | 0.872     | 0.735  | 0.798    |
|                 | SVM            | 0.826    | 0.807     | 0.902  | 0.852    |
|                 | Neural Network | 0.821    | 0.829     | 0.853  | 0.841    |
| <i>Med 40%</i>  | Decision Tree  | 0.777    | 0.851     | 0.725  | 0.783    |
|                 | SVM            | 0.832    | 0.826     | 0.882  | 0.853    |
|                 | Neural Network | 0.821    | 0.829     | 0.853  | 0.841    |
| <i>High 55%</i> | Decision Tree  | 0.745    | 0.789     | 0.735  | 0.761    |
|                 | SVM            | 0.837    | 0.821     | 0.902  | 0.860    |
|                 | Neural Network | 0.832    | 0.832     | 0.873  | 0.852    |

Tabella 10: Metriche di performance dei modelli su diversi livelli di rumore

La Tabella 10 fornisce un quadro quantitativo e comparativo del degrado prestazionale dei tre modelli all'aumentare dei dati mancanti. Analizzando le metriche di Accuracy, Precision, Recall e F1-Score, emergono dinamiche comportamentali nettamente distinte tra le architetture testate.

In primo luogo, si rileva l'estrema vulnerabilità del Decision Tree. Partendo da una Baseline già inferiore agli altri modelli (Accuracy 0.804), l'algoritmo subisce un crollo progressivo e significativo, scendendo a un'Accuracy di 0.745 al livello di rumore estremo (55%). Il dato più critico riguarda la sua Recall, che precipita da 0.853 a 0.735 già con solo il 15% di rumore. Questo indica che, in presenza di dati imputati, l'albero di decisione diventa un modello "iper-cauto", aumentando temporaneamente la Precision (0.872 al 15%) a scapito della capacità di individuare correttamente i casi positivi (Falsi Negativi), un difetto grave in ambito diagnostico.

Al contrario, la Support Vector Machine (SVM) si conferma come il modello più robusto dell'intero esperimento. Mantiene il primato in tutte le fasi di test: la sua Accuracy oscilla in modo marginale (da 0.842 della Baseline a 0.837 nel livello High), dimostrando una quasi totale indifferenza al rumore. Ancora più rilevante è il mantenimento di una Recall costantemente superiore a 0.88 (raggiungendo 0.902 al 55%), confermando che l'uso del kernel RBF permette di preservare la corretta classificazione dei pazienti malati anche quando oltre metà delle feature viene ricostruita artificialmente.

Infine, la Neural Network evidenzia un comportamento molto stabile e bilanciato. Partendo da ottimi valori iniziali (Accuracy 0.832 e Recall 0.882), la rete gestisce il degrado informativo con notevole resilienza, stabilizzandosi al livello High (55%) su un F1-Score di 0.852, praticamente identico alla Baseline (0.853). Ciò dimostra che la natura distribuita dei pesi sinaptici compensa efficacemente i valori mediani introdotte dal processo di imputazione.

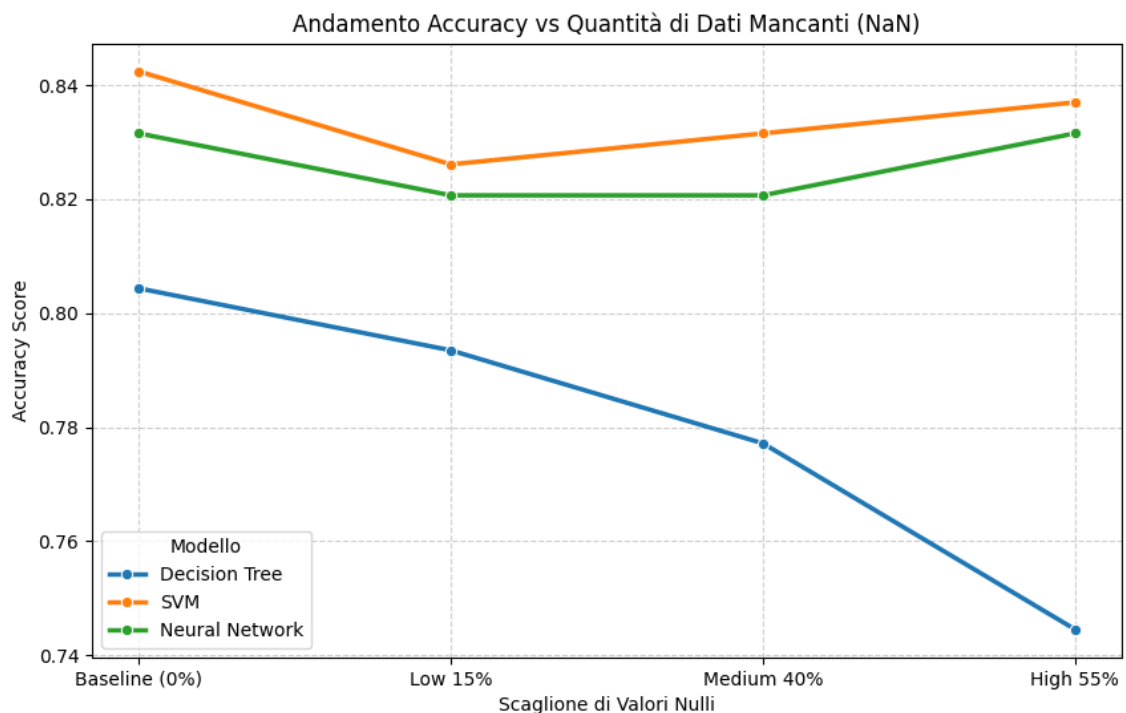


Figura 17: Il grafico mostra l'andamento dell'accuracy al aumentare dei valori nulli per gli algoritmi testati Decision Tree, SVM e Reti Neurali

A conferma di quanto detto finora, osserviamo nella figura 17 come la curva che rappresenta il decision tree ha un netto crollo di accuratezza all'aumentare dei valori

nulli, maggiormente marcato nel momento in cui si superamento del 40%. Mentre i metodi SVM e Reti Neurali hanno un comportamento simile, dove la curva del SVM parte dalla accuratezza più elevata dei tre e tende ad essere piatta nonostante un piccolo calo iniziale tra il 0% e il 15%. In fine, La curva della Rete Neurale si mostra stabile e bilanciata.

### 8.1.1 Cross-Validation

Per poter validare i risultati analizzati fin ora, e quindi confermare la validità statistica dei valori e assicurare che le performance dei modelli non siano dipendenti da una specifica e fortuita partizione del dataset (train-test split), è stata applicata una tecnica di K-Fold Cross-Validation (con  $k = 5$ ).

Analizzando i risultati medi dell'accuracy nei diversi fold, si ha la prova concreta di quanto un modello sia solido o meno.

La Tabella 11 riporta l'accuracy media di cross-validation al variare del livello di valori mancanti, con preprocessing di base (imputazione mediana + scaling).

| Livello NaN   | Decision Tree | SVM    | Neural Network |
|---------------|---------------|--------|----------------|
| Baseline (0%) | 0.7541        | 0.8180 | 0.8071         |
| Low 15%       | 0.7296        | 0.7976 | 0.7962         |
| Medium 40%    | 0.7038        | 0.7609 | 0.7731         |
| High 55%      | 0.7161        | 0.6930 | 0.7405         |

Tabella 11: Confronto dell'Accuracy Media CV al variare della percentuale di valori mancanti (Strategia: Base SimpleImputer)

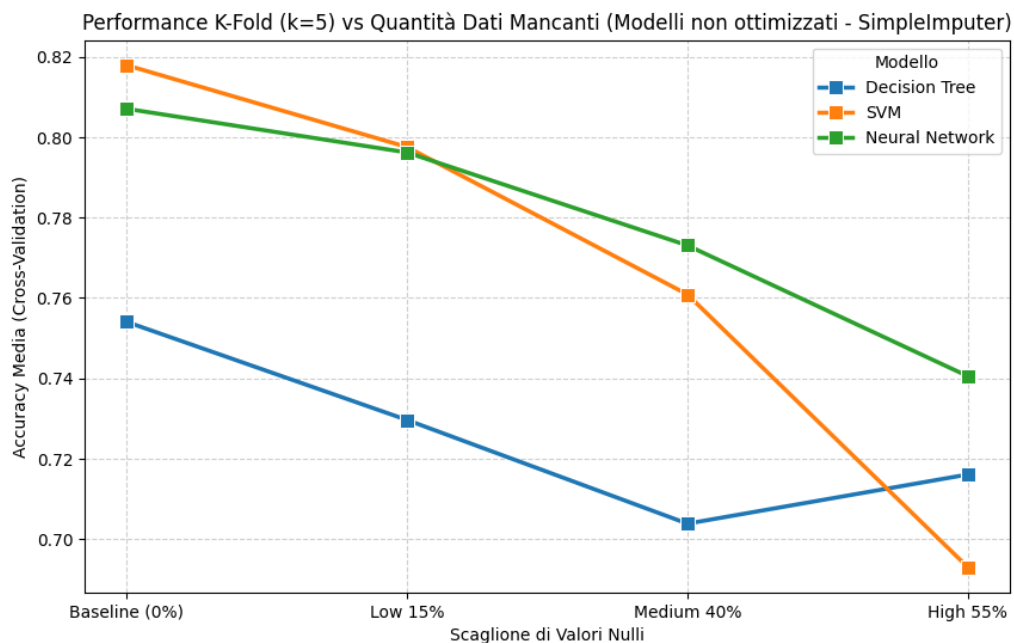


Figura 18: Il grafico mostra l'andamento dell'accuracy eseguendo la cross validation k-fold,  $k = 5$ , con gli algoritmi non ottimizzati e con l'imputer basico



La K-Fold Cross-Validation a 5 split ribalta i risultati iniziali, svelando la vera vulnerabilità dei modelli al rumore estremo (55%).

La SVM subisce un drastico ridimensionamento: le ottime performance del singolo split erano dovute al caso (high variance). Nella validazione incrociata le prestazioni precipitano, poiché l'uso notevole della mediana azzerava la varianza locale, compromettendo l'iperpiano del kernel RBF. La Neural Network si conferma invece il modello più solido, mantenendo un'Accuracy media superiore a 0.74. A differenza della SVM, l'aggiornamento dei suoi pesi sinaptici tollera efficacemente l'appiattimento informativo causato dall'imputazione semplice.

In conclusione, dal confronto validato tramite Cross-Validation è emersa una chiara gerarchia. Il Decision Tree si è confermato strutturalmente inadeguato a gestire la sparsità informativa, risultando il modello più vulnerabile/fragile. La SVM, inizialmente apparsa come il modello più resiliente, ha rivelato una profonda instabilità geometrica di fronte alla sostituzione dei valori nulli con la mediana della rispettiva colonna, specialmente negli scenari ad alto rumore (55%). Il modello che ha ottenuto la miglior performance in questo test di robustezza è risultato essere la Neural Network. In assenza di ottimizzazioni avanzate e di imputazioni multivariate, il modello neurale ha offerto la maggiore affidabilità predittiva, assorbendo le anomalie dei dati ricostruiti grazie alla natura distribuita delle sue funzioni di attivazione.

## 8.2 Valori duplicati

Analogamente alle altre tipologie di rumore, i tre modelli sono stati riaddestrati sul training set con duplicati e valutati sul medesimo test set pulito. Le matrici di confusione per i quattro livelli sono riportate nelle Figure 19–22.

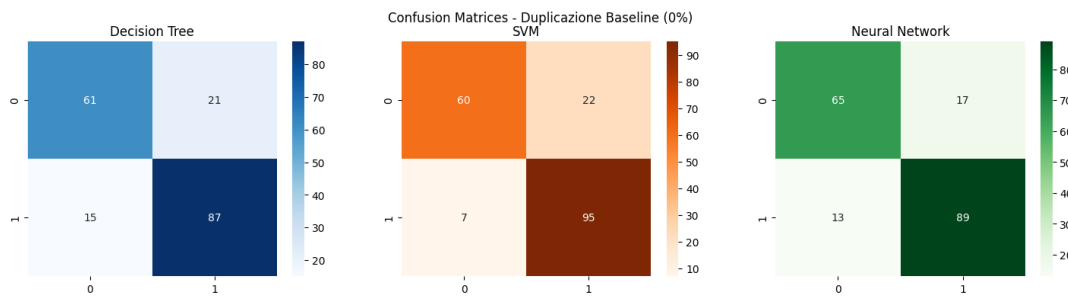


Figura 19: Matrice di confusione con il training set originale (Baseline, 0% duplicati)

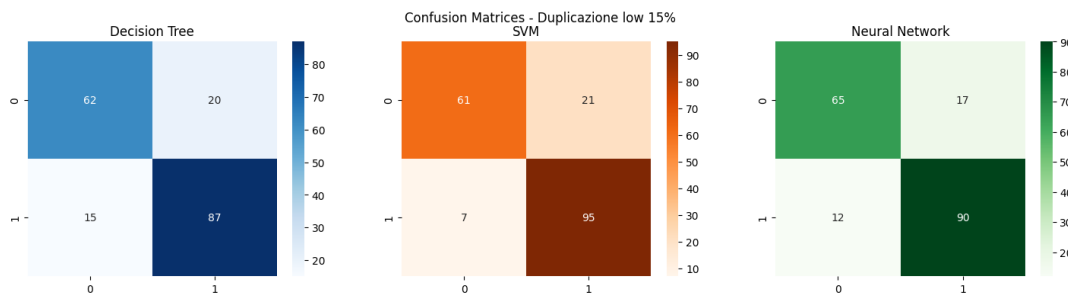


Figura 20: Matrice di confusione con il 15% di righe duplicate

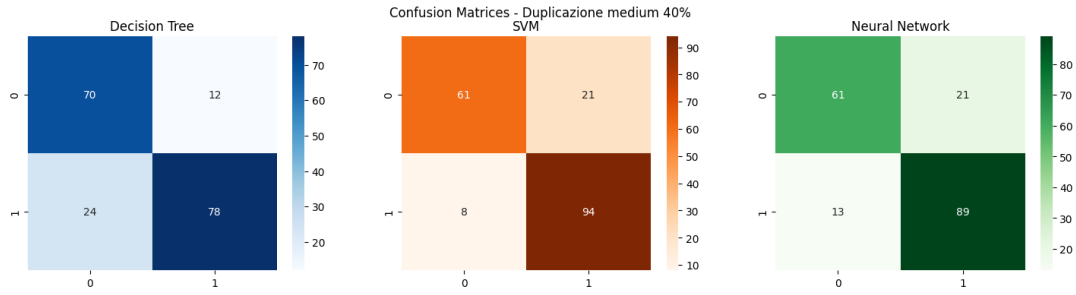


Figura 21: Matrice di confusione con il 40% di righe duplicate

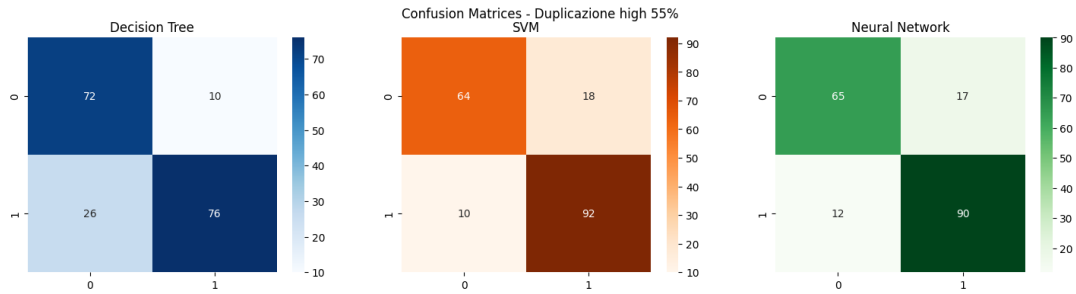


Figura 22: Matrice di confusione con il 55% di righe duplicate

A differenza di quanto osservato per i valori mancanti e per i dati sporchi, la presenza di righe duplicate produce un effetto pressoché trascurabile sul test set pulito. Il Decision Tree mantiene 15 Falsi Negativi tra Baseline e Low, per poi salire a 24 e 26 rispettivamente ai livelli Medium e High; la SVM passa da 7 (Baseline/Low) a 8 (Medium) e 10 (High) Falsi Negativi; la Neural Network resta stabile tra 12 e 13 casi in tutti gli scenari. Questo comportamento è coerente con la natura del rumore introdotto: duplicare una riga non aggiunge informazione nuova, ma ne aumenta soltanto il peso relativo nella funzione di perdita durante l'addestramento.

| Livello       | Modello        | Accuracy | Precision | Recall | F1-Score |
|---------------|----------------|----------|-----------|--------|----------|
| Baseline (0%) | Decision Tree  | 0.804    | 0.806     | 0.853  | 0.829    |
|               | SVM            | 0.842    | 0.812     | 0.931  | 0.868    |
|               | Neural Network | 0.837    | 0.840     | 0.873  | 0.856    |
| Low (15%)     | Decision Tree  | 0.810    | 0.813     | 0.853  | 0.833    |
|               | SVM            | 0.848    | 0.819     | 0.931  | 0.872    |
|               | Neural Network | 0.842    | 0.841     | 0.882  | 0.861    |
| Medium (40%)  | Decision Tree  | 0.804    | 0.867     | 0.765  | 0.813    |
|               | SVM            | 0.842    | 0.817     | 0.922  | 0.866    |
|               | Neural Network | 0.815    | 0.809     | 0.873  | 0.840    |
| High (55%)    | Decision Tree  | 0.804    | 0.884     | 0.745  | 0.809    |
|               | SVM            | 0.848    | 0.836     | 0.902  | 0.868    |
|               | Neural Network | 0.842    | 0.841     | 0.882  | 0.861    |

Tabella 12: Metriche di performance sul test set, ai vari livelli di duplicazione del training set.

L'unico modello a mostrare un degrado apprezzabile è il Decision Tree, la cui Recall scende da 0.853 (baseline) a 0.745 (High 55%). La SVM mantiene un F1-Score compreso tra 0.866 e 0.872 in tutti gli scenari; la Neural Network oscilla in un intervallo leggermente più ampio (0.840–0.861), con un minimo al livello Medium, senza però mostrare alcun trend di degrado sistematico.

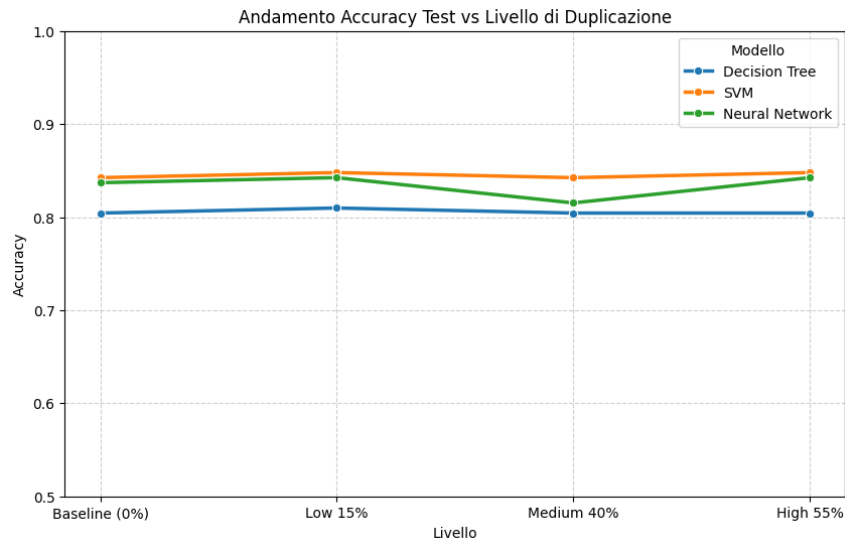


Figura 23: Andamento dell'Accuracy sul test set al variare del livello di duplicazione

A differenza delle curve nette osservate per i valori mancanti e per i dati sporchi, le curve in Figura 23 restano pressoché piatte, con oscillazioni contenute entro 1–2 punti percentuali. Questo risultato è importante da confrontare con quanto emerge dalla Cross-Validation nella sotto-sezione seguente.

### 8.2.1 Cross-Validation

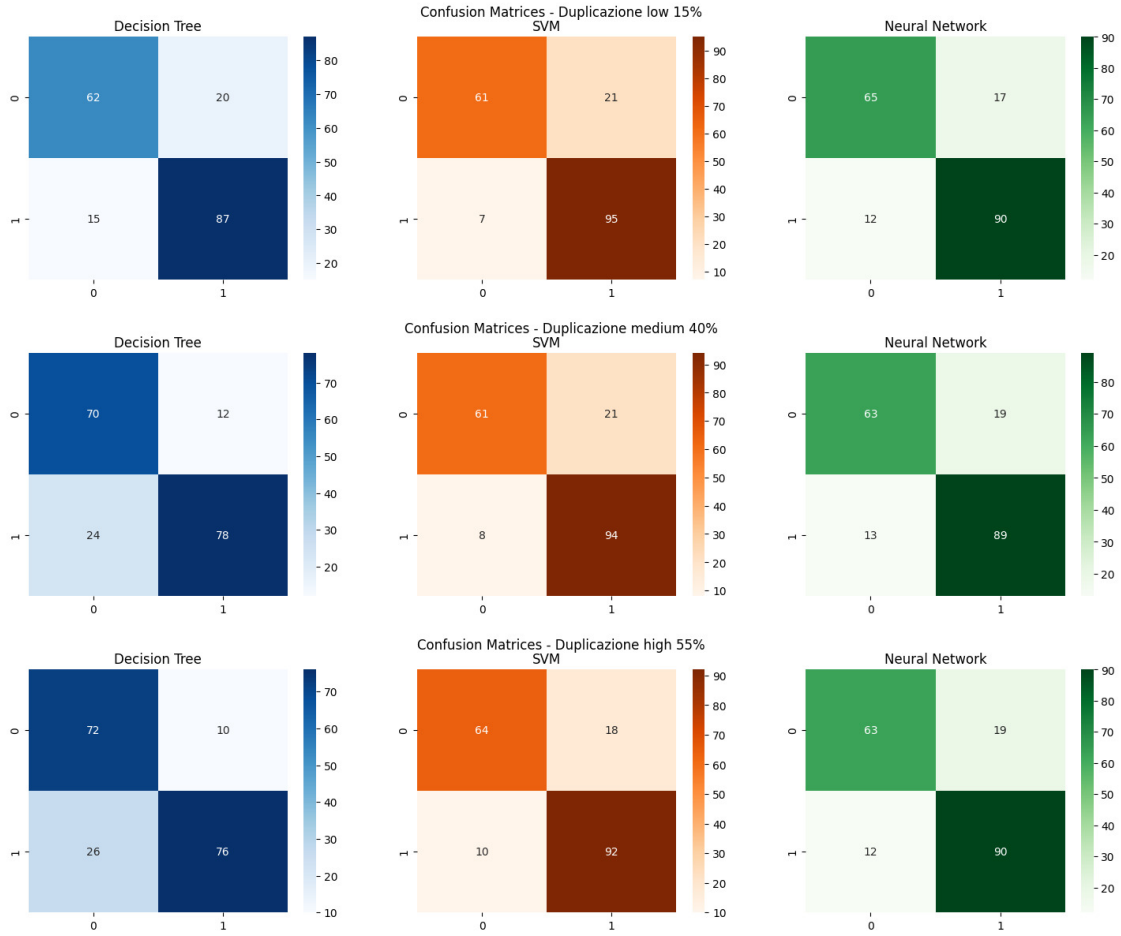


Figura 24: Confronto dell'Accuracy tra i modelli di baseline

| Livello       | Decision Tree | SVM   | Neural Network |
|---------------|---------------|-------|----------------|
| Baseline (0%) | 0.754         | 0.818 | 0.807          |
| Low (15%)     | 0.784         | 0.830 | 0.811          |
| Medium (40%)  | 0.777         | 0.847 | 0.824          |
| High (55%)    | 0.799         | 0.854 | 0.823          |

Tabella 13: Accuracy media di CV (5-fold) al variare del livello di duplicazione (dati non ripuliti).

A differenza delle altre tipologie di rumore, l'accuracy in cross-validation **aumenta** apparentemente con il livello di duplicazione. Questo risultato è fuorviante: è dovuto a un fenomeno di *data leakage*, poiché la suddivisione in fold della cross-validation può separare copie identiche della stessa riga tra training e validation del fold, rendendo la stima della performance eccessivamente ottimistica.

Il confronto diretto tra la Tabella 12 (valutazione onesta sul test set) e la Tabella 13 (valutazione in Cross-Validation) permette di isolare due dinamiche:

- **Il segnale reale è piatto:** sul test set indipendente l'Accuracy resta sostanzialmente invariata al crescere della duplicazione per tutti e tre i modelli, a conferma che duplicare righe non introduce né informazione utile né rumore dannoso;
- **Il segnale in CV è un artefatto, non legato alla complessità del modello:** tra Baseline e High 55% l'accuracy di CV cresce di +4.5 punti percentuali per il Decision Tree (0.754→0.799), +3.6 per la SVM (0.818→0.854) e +1.6 per la Neural Network (0.807→0.823). Nessuno di questi incrementi trova riscontro in Tabella 12: il fatto che sia interessato anche un modello semplice e regolarizzato come il Decision Tree dimostra che il leakage è una proprietà dello split dei dati, non dell'algoritmo, e va quindi corretto a monte (Sezione 9.2), non tramite tecniche di regolarizzazione del modello.

### 8.3 Dati sporchi

In questa sezione analizziamo il comportamento dei 3 modelli (SVM, Decision Tree, e Reti Neurali) con l'introduzione di valori sporchi sempre come visto nei precedenti casi a scaglioni. In questa fase di test introducendo valori sbagliati il rischio per gli algoritmi è quello di apprendere pattern errati.

Di seguito, analizziamo le matrici di confusione per ogni livello di valori sporchi aggiunti, per analizzare il livello di degrado che viene raggiunto dagli algoritmi e quale di esso abbia il comportamento migliore.

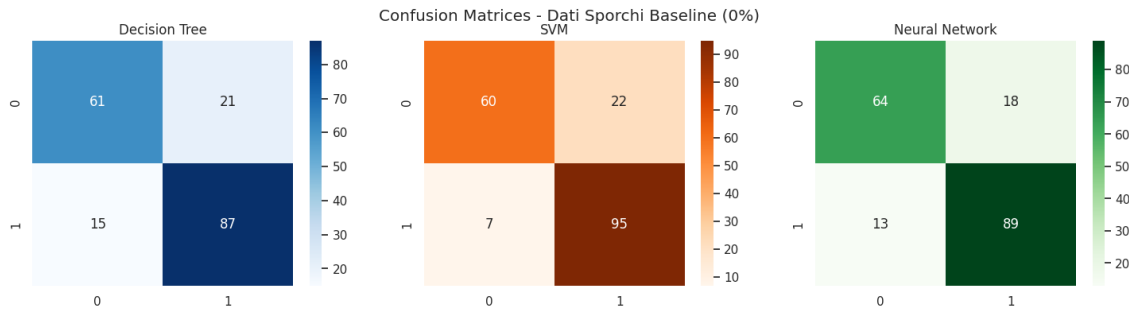


Figura 25: Matrice di confusione per i tre modelli di ML senza valori sporchi aggiunti

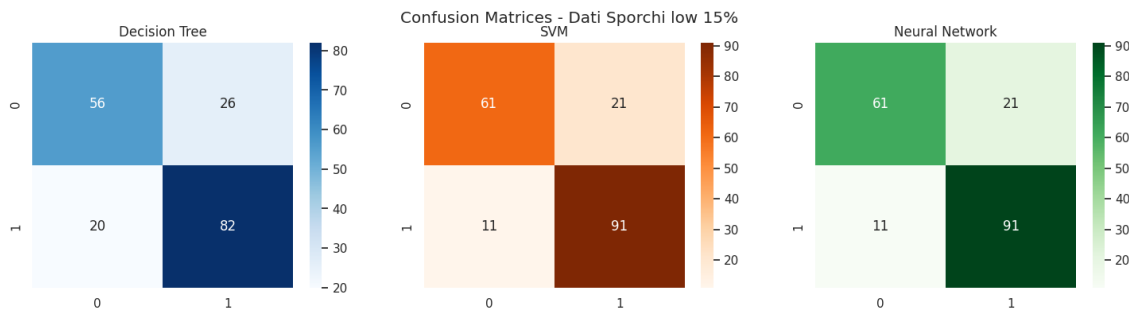


Figura 26: Matrice di confusione per i tre modelli di ML con il 15% di valori sporchi aggiunti

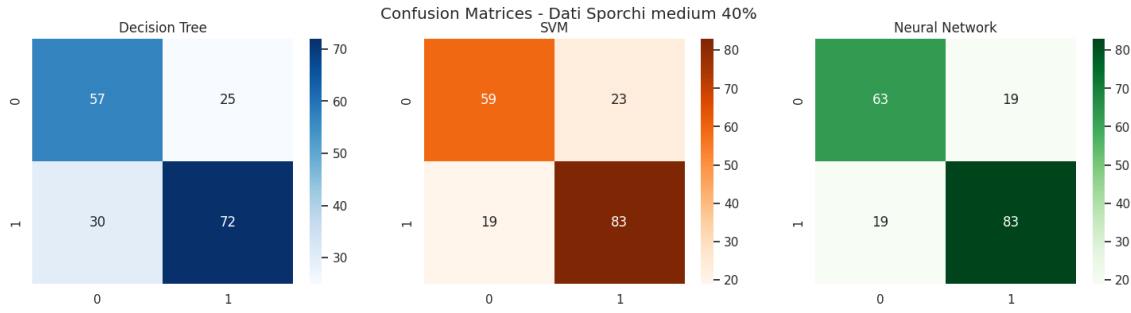


Figura 27: Matrice di confusione per i tre modelli di ML con il 40% di valori sporchi aggiunti

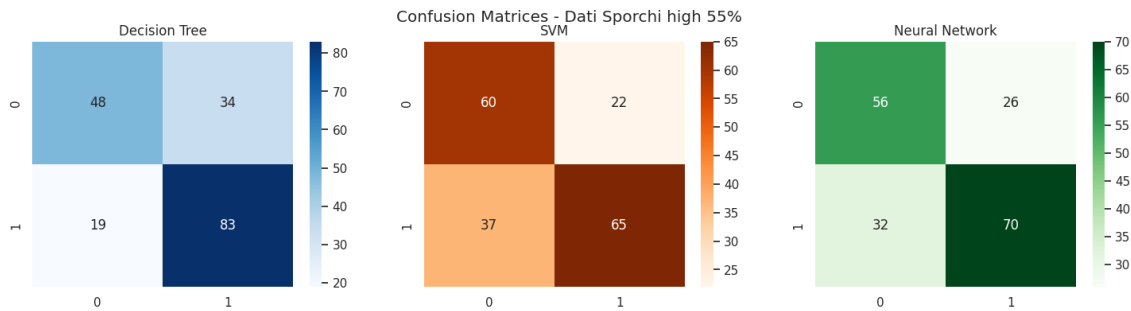


Figura 28: Matrice di confusione per i tre modelli di ML con il 55% di valori sporchi aggiunti

La Figura 25 mostra il comportamento degli algoritmi con il dataset pulito; questa serie di matrici rappresenta la baseline di riferimento che utilizzeremo per valutare il livello di degrado al crescere della percentuale di valori sporchi (rumore) inseriti nel dataset.

Con l'introduzione del 15% di valori sporchi, come mostrato nella Figura 26, si può notare che la Rete Neurale e la SVM dimostrano un'ottima resilienza iniziale, presentando matrici di confusione pressoché identiche a quelle della baseline. Al contrario, il Decision Tree mostra già un primo calo delle prestazioni, passando da 15 a 20 Falsi Negativi.

Aumentando la percentuale di dati sporchi al 40% (Figura 27), il Decision Tree evidenzia forti difficoltà nella classificazione, raggiungendo i 30 Falsi Negativi. I modelli SVM e Rete Neurale, seppur comportandosi decisamente meglio rispetto all'albero di decisione, subiscono un raddoppio delle classificazioni errate di tipo Falso Negativo. Questo andamento conferma che, a questo livello di corruzione, l'addestramento inizia a interiorizzare i falsi pattern causati dai valori sporchi, i quali distorcono lo spazio vettoriale e l'aggiornamento dei pesi sinaptici.

Infine, raggiungendo il 55% di valori sporchi (superando quindi la quantità di dati reali), si osserva un drastico decadimento prestazionale, come visibile nella Figura 28. La SVM subisce un ulteriore raddoppio dei Falsi Negativi, dimostrando che il rumore ha alterato le distanze vettoriali a tal punto da rendere il kernel RBF incapace

di tracciare un margine di separazione sensato, portando a misclassificare oltre un terzo dei pazienti malati. Anche la Rete Neurale subisce un grave peggioramento, toccando i 32 Falsi Negativi: l'eccesso di rumore satura i gradienti durante l'addestramento, impedendo al modello di distinguere il segnale clinico dalle anomalie. Il Decision Tree, invece, registra un comportamento anomalo: i Falsi Negativi si riducono a 19, ma si assiste a un'esplosione dei Falsi Positivi. Questo squilibrio è riconducibile alla generazione di regole di split caotiche che, nel tentativo di mitigare l'elevata incertezza dei valori, finiscono per favorire la classe positiva in modo fuorviante.

| Livello              | Modello        | Accuracy | Precision | Recall | F1-Score |
|----------------------|----------------|----------|-----------|--------|----------|
| <i>Baseline (0%)</i> | Decision Tree  | 0.804    | 0.806     | 0.853  | 0.829    |
|                      | SVM            | 0.842    | 0.812     | 0.931  | 0.868    |
|                      | Neural Network | 0.832    | 0.832     | 0.873  | 0.852    |
| <i>Low 15%</i>       | Decision Tree  | 0.750    | 0.759     | 0.804  | 0.781    |
|                      | SVM            | 0.826    | 0.812     | 0.892  | 0.850    |
|                      | Neural Network | 0.826    | 0.812     | 0.892  | 0.850    |
| <i>Medium 40%</i>    | Decision Tree  | 0.701    | 0.742     | 0.706  | 0.724    |
|                      | SVM            | 0.772    | 0.783     | 0.814  | 0.798    |
|                      | Neural Network | 0.793    | 0.814     | 0.814  | 0.814    |
| <i>High 55%</i>      | Decision Tree  | 0.712    | 0.709     | 0.814  | 0.758    |
|                      | SVM            | 0.679    | 0.747     | 0.637  | 0.688    |
|                      | Neural Network | 0.685    | 0.729     | 0.686  | 0.707    |

Tabella 14: Metriche di performance dei modelli ai vari livelli di corruzione del dataset (Dati Sporchi).

La Tabella 14 quantifica il degrado causato dall'inserimento di dati sporchi nel set di addestramento, evidenziando due principalmente due situazioni:

- La SVM e la Rete Neurale nonostante sono stabili al 15% di rumore (F1-Score 0.850), le prestazioni precipitano al 55%. È particolarmente critico il peggioramento della Recall per la SVM (da 0.931 a 0.637): a causa dei falsi pattern, il modello perde quasi il 30% della sua sensibilità diagnostica.
- Il Decision Tree come visto prima Al 55% di rumore mostra un'anomalia dove l'albero registra una Recall elevata (0.814) accompagnata però dalla peggior Precision del test (0.709). Per gestire i dati sporchi, l'algoritmo sovra-prevede sistematicamente la classe positiva, generando un eccesso di Falsi Positivi che ne azzerà l'affidabilità clinica.

Le dinamiche finora descritte trovano perfetta corrispondenza grafica nella Figura 29. È evidente il crollo verticale delle prestazioni per SVM e Rete Neurale non appena il rumore supera il 15%. Risulta inoltre ben visibile l'anomala risalita della curva del Decision Tree nello scenario più estremo (55%): come dimostrato dai dati tabellari (tabella 14, questo 'miglioramento' dell'Accuracy è falso, frutto di uno sbilanciamento predittivo che non garantisce alcuna reale affidabilità clinica.

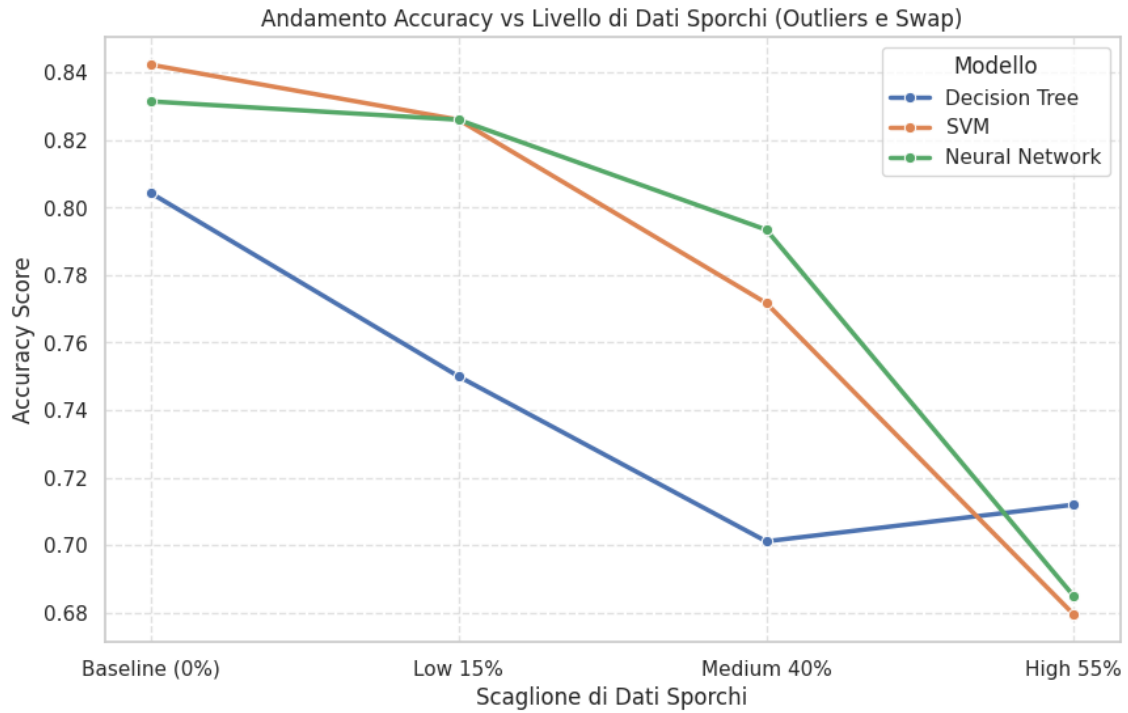


Figura 29: Grafico sul accuratezza al aumentare dei valori sporchi

### 8.3.1 Cross Validation

A questo punto per avere la certezza che l'analisi sia solida e che i risultati non siano frutto di un singolo train-test split particolarmente sbilanciato, si applica la K-Fold Cross-Validation a 5 split. Guardando le medie dell'Accuracy, emerge chiaramente quanto i modelli base siano vulnerabili di fronte all'inserimento di dati falsati. I numeri sono riassunti nella tabella 15 sottostante:

| Livello       | Decision Tree | SVM   | Neural Network |
|---------------|---------------|-------|----------------|
| Baseline (0%) | 0.754         | 0.818 | 0.808          |
| Low 15%       | 0.731         | 0.766 | 0.761          |
| Medium 40%    | 0.693         | 0.683 | 0.694          |
| High 55%      | 0.647         | 0.697 | 0.702          |

Tabella 15: Accuracy media in K-Fold Cross-Validation (K=5) per ciascun modello e livello di corruzione.

Di seguito vengono mostrati i dati della tabella 15 sotto forma grafica.



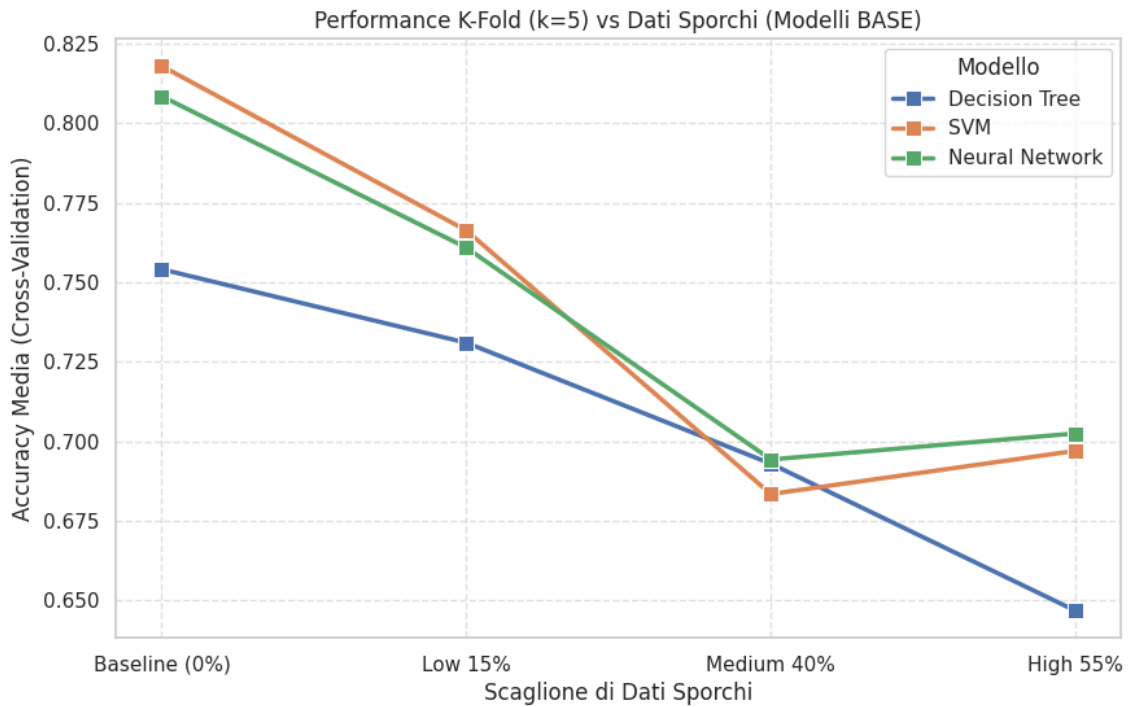


Figura 30: Andamento della Accuracy per ciascun modello tramite Cross-Validation k-fold

L'analisi di queste misurazioni permette di evidenziare tre dinamiche fondamentali:

- Il risultato più importante della Cross-Validation si ottiene osservando l'andamento dell'albero di decisione, dove il finto miglioramento notato nel singolo test al 55% scompare del tutto. Calcolando la media sui 5 fold, l'Accuracy precipita al valore più basso dell'intero esperimento (0.647). Questo dimostra che l'algoritmo non sa adattarsi al rumore, ma si limita a creare regole di classificazione instabili e prive di logica.
- A differenza di quanto visto con i dati mancanti, l'inserimento di dati "sporchi" ha un effetto distruttivo molto più rapido. Già al 40% di rumore, tutti i modelli crollano sotto la soglia dello 0.70. Questo accade perché gli algoritmi apprendono dei pattern falsi, i quali causano un forte disturbo nella fase di addestramento, alterando i gradienti (per la Rete Neurale) e i vettori di supporto (per la SVM).
- Nello scenario peggiore, dove è presente il 55% di valori corrotti, la Rete Neurale (0.702) e la SVM (0.697) ottengono risultati quasi identici, fermandosi su livelli di affidabilità del tutto inaccettabili. Tuttavia, la Rete Neurale mantiene un minimo vantaggio rispetto agli altri modelli.

In conclusione, questo test dimostra una regola fondamentale: nessun modello, per quanto avanzato, può formulare una predizione corretta se più della metà delle informazioni che gli vengono fornite è corrotta. Pertanto, questa tipologia di rumore risulta la più dannosa per le prestazioni. Tutti e tre i modelli mostrano un calo consistente e progressivo dell'Accuracy all'aumentare del livello di rumore, poiché i valori errati alterano direttamente la relazione logica tra feature e target senza essere facilmente rilevabili e isolabili, come avviene invece nel caso dei valori nulli.

## 9 Tecniche di preprocessing e regolarizzazione per la mitigazione del rumore

Per ciascuna tipologia di rumore sono state applicate tecniche specifiche di preprocessing e/o regolarizzazione dei modelli, al fine di valutarne l'efficacia nel contenere il degrado delle prestazioni osservato nella sezione precedente.

### 9.1 Mitigazione dei valori mancanti

Oltre all'imputazione di base con la mediana, sono state testate due strategie di imputazione più avanzate, in combinazione con tecniche di regolarizzazione dei modelli (Decision Tree con `max_depth=4`, `min_samples_split=20`, `min_samples_leaf=15`; SVM con margine più "morbido",  $C = 0.1$ ; rete neurale con Dropout (0.3 e 0.2) e EarlyStopping):

- **KNN Imputer**: imputa ciascun valore mancante sulla base della media dei  $k = 5$  vicini più simili;
- **Iterative Imputer**: stima ogni valore mancante modellandolo come funzione delle altre variabili, in modo iterativo (`max_iter=10`).

#### 9.1.1 Imputazione con KNN

L'applicazione del KNN Imputer, insieme all'introduzione di vincoli di regolarizzazione per prevenire l'overfitting sui dati ricostruiti, ha generato un impatto di miglioramento sulle performance architetturali su tutti i 3 gli algoritmi utilizzati, i quali sono visibili nell'andamento dell'Accuracy (Figura 31) e confermati dai valori in Tabella ??.

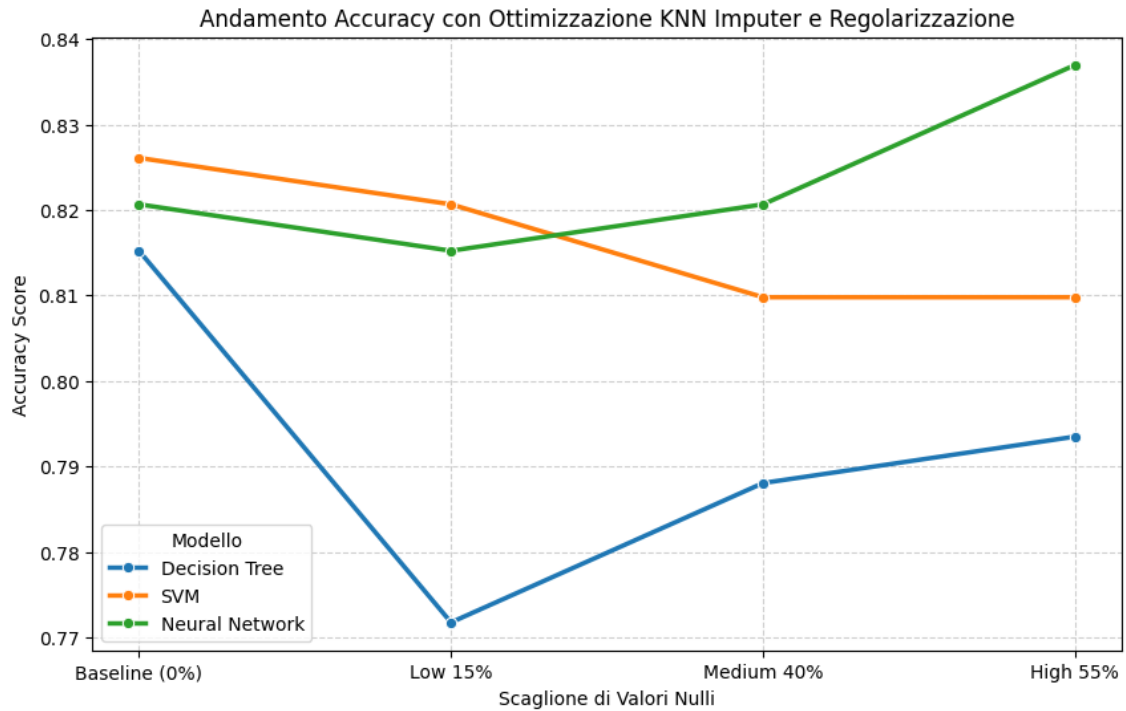


Figura 31: Andamento dell'Accuracy con ottimizzazione KNN Imputer e Regolarizzazione.

| Livello              | Modello        | Accuracy | Precision | Recall | F1-Score |
|----------------------|----------------|----------|-----------|--------|----------|
| <i>Baseline (0%)</i> | Decision Tree  | 0.815    | 0.815     | 0.863  | 0.838    |
|                      | SVM            | 0.826    | 0.792     | 0.931  | 0.856    |
|                      | Neural Network | 0.815    | 0.809     | 0.873  | 0.840    |
| <i>Low 15%</i>       | Decision Tree  | 0.772    | 0.724     | 0.951  | 0.822    |
|                      | SVM            | 0.821    | 0.790     | 0.922  | 0.851    |
|                      | Neural Network | 0.821    | 0.829     | 0.853  | 0.841    |
| <i>Medium 40%</i>    | Decision Tree  | 0.788    | 0.812     | 0.804  | 0.808    |
|                      | SVM            | 0.810    | 0.777     | 0.922  | 0.843    |
|                      | Neural Network | 0.810    | 0.819     | 0.843  | 0.831    |
| <i>High 55%</i>      | Decision Tree  | 0.793    | 0.814     | 0.814  | 0.814    |
|                      | SVM            | 0.810    | 0.772     | 0.931  | 0.844    |
|                      | Neural Network | 0.826    | 0.837     | 0.853  | 0.845    |

Tabella 16: Metriche di performance per i modelli ottimizzati con KNN Imputer e Regolarizzazione.

L'analisi di queste metriche evidenzia tre dinamiche fondamentali:

- L'adozione del KNN Imputer e di un soft margin ( $C = 0.1$ ) ha permesso alla SVM di mantenere un'ottima stabilità prestazionale, registrando un'Accuracy dello 0.810 anche nello scenario più critico (55%). Il dato più rilevante in ottica clinica è la *Recall* (Sensibilità), che si attesta a un eccellente 0.931. Come confermato dalla matrice di confusione (Figura 35), l'imputazione basata sulla distanza dei vicini ha preservato la coerenza geometrica dello spazio

delle feature, permettendo al kernel RBF di tracciare un iperpiano stabile che commette appena 7 Falsi Negativi su 102 pazienti positivi reali.

- La Rete Neurale si conferma il modello con la migliore capacità di generalizzazione complessiva, raggiungendo nello scenario peggiore ovvero il 55% l'Accuracy più elevata dell'intero test (0.826) e una F1-Score di 0.845. L'introduzione del Dropout e dell'EarlyStopping ha impedito efficacemente all'architettura di memorizzare il rumore introdotto dalla ricostruzione dei dati. Si delinea così un interessante compromesso diagnostico: se la Rete Neurale offre la massima precisione globale, la SVM garantisce la massima sicurezza nel ridurre al minimo i casi di mancata diagnosi (Falsi Negativi).
- Sebbene l'albero di decisione si confermi il modello meno performante del gruppo, la potatura applicata tramite i parametri `max_depth` e `min_samples_leaf` ne ha modificato il comportamento. A differenza del crollo continuo e verticale registrato in assenza di mitigazione, l'algoritmo regolarizzato presenta un crollo iniziale al 15% di valori nulli, per poi recuperare e stabilizzarsi in modo resiliente su un'Accuracy vicina allo 0.79 negli scenari a maggiore corruzione (raggiungendo lo 0.793 al 55%). Questo dimostra che i vincoli strutturali impediscono all'albero di degenerare, garantendogli una soglia prestazionale di sicurezza.

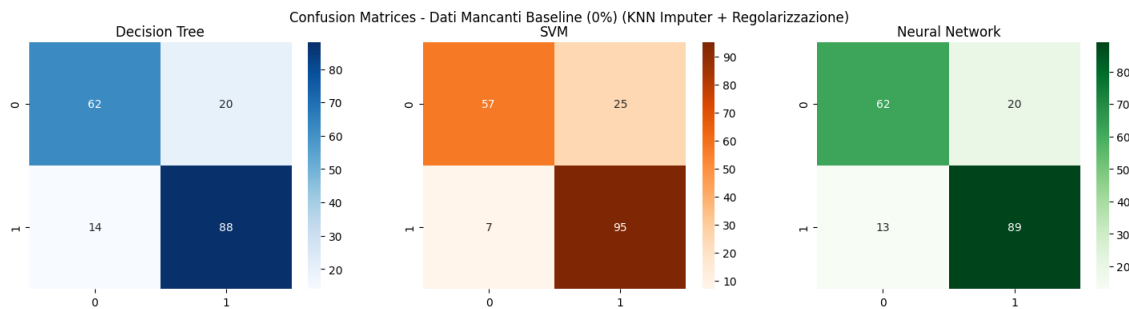


Figura 32: Matrice di confusione con imputazione KNN e algoritmi regolarizzati (Baseline)

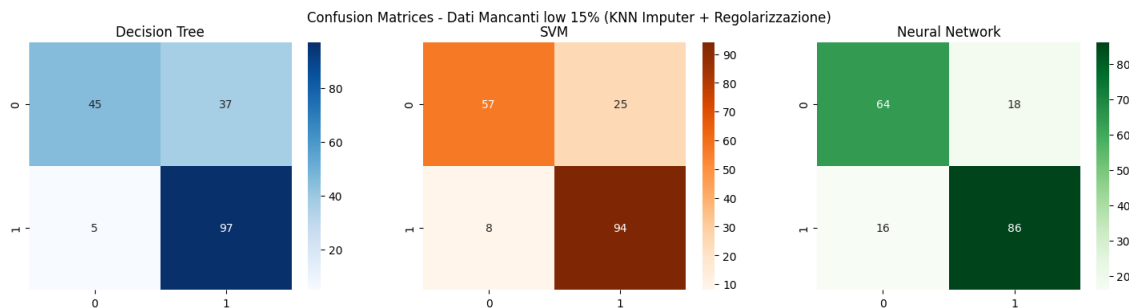


Figura 33: Matrice di confusione con imputazione KNN e algoritmi regolarizzati (15% di valori nulli)

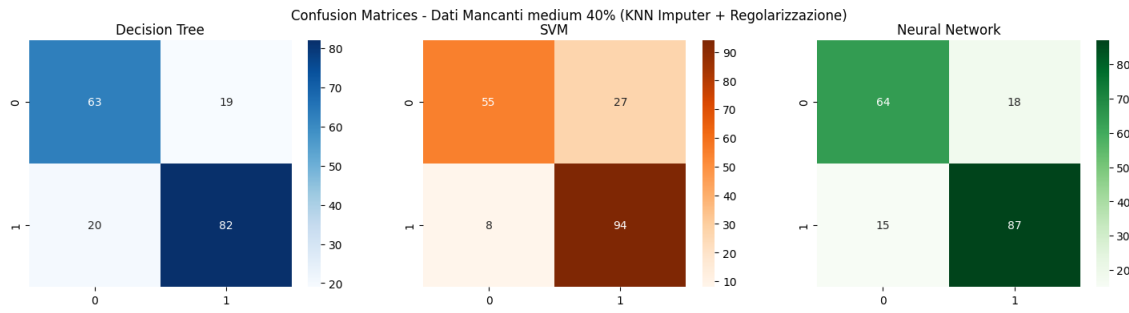


Figura 34: Matrice di confusione con imputazione KNN e algoritmi regolarizzati (40% di valori nulli)

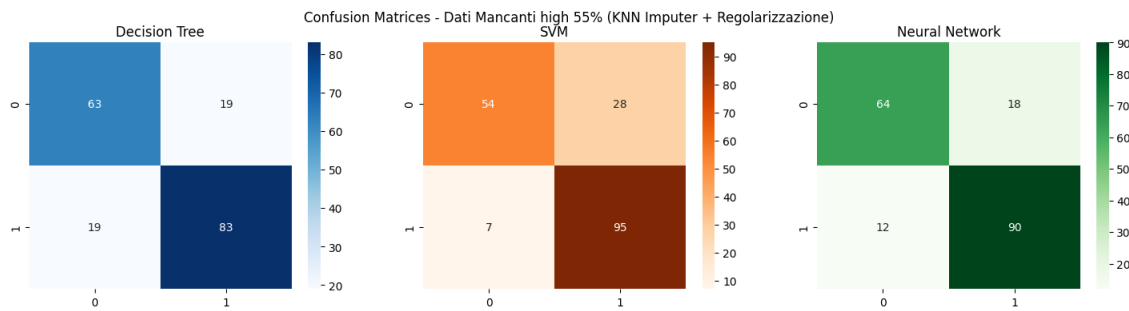


Figura 35: Matrice di confusione con imputazione KNN e algoritmi regolarizzati (55% di valori nulli)

Per mitigare il bias di varianza del singolo partizionamento e ottenere una stima reale della capacità di generalizzazione, i modelli ottimizzati sono stati sottoposti a una procedura di K-Fold Cross-Validation (K=5).

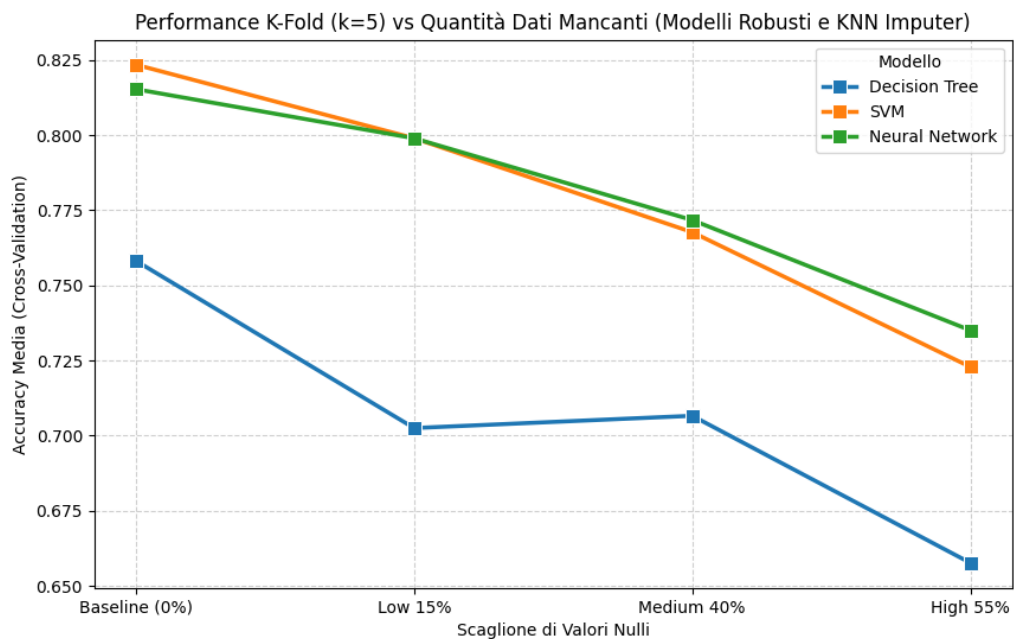


Figura 36: Andamento dell'Accuracy media in Cross-Validation (K=5) post-mitigazione con KNN Imputer.

| <b>Livello Dati Mancanti</b> | <b>Decision Tree</b> | <b>SVM</b> | <b>Neural Network</b> |
|------------------------------|----------------------|------------|-----------------------|
| Baseline (0%)                | 0.758                | 0.823      | 0.802                 |
| Low 15%                      | 0.702                | 0.799      | 0.799                 |
| Medium 40%                   | 0.707                | 0.768      | 0.772                 |
| High 55%                     | 0.658                | 0.723      | 0.723                 |

Tabella 17: Accuracy media in K-Fold CV (K=5) per i modelli ottimizzati con KNN Imputer.

L'analisi dell'Accuracy media calcolata sui cinque fold (Tabella 17 e Figura 36) ridimensiona i valori ottenuti dal singolo test split, offrendo una prospettiva più realistica e rigorosa. Nello specifico, emergono le seguenti dinamiche:

- A differenza della tenuta quasi perfetta registrata nel singolo split, la validazione incrociata evidenzia un degrado fisiologico e progressivo per tutte le architetture. Questo conferma che l'imputazione multivariata, per quanto avanzata, non può ricreare dal nulla l'informazione strutturale perduta: all'aumentare della *data sparsity*, il rumore artificiale introdotto dal KNN finisce inevitabilmente per erodere una parte del segnale clinico utile.
- Nonostante il calo prestazionale, l'approccio combinato di KNN Imputer e regolarizzazione dimostra una buona efficacia nell'evitare il degrado strutturale dei modelli. Nello scenario di degradazione estrema (55%), la Support Vector Machine e la Rete Neurale mostrano traiettorie di decadimento quasi sovrapponibili, assestandosi entrambe su un'Accuracy media di 0.723. Si tratta di un degrado controllato che mantiene le previsioni significativamente al di sopra della soglia di casualità, confermando la robustezza delle due architetture.
- L'albero di decisione si conferma l'algoritmo intrinsecamente più vulnerabile alla perdita di dati. Pur beneficiando dei parametri di potatura, la sua natura gerarchica lo porta a soffrire maggiormente l'imputazione artificiale, registrando l'Accuracy più bassa in ogni scenario e chiudendo la sperimentazione al 55% di valori nulli con una media di appena 0.658.

### 9.1.2 Imputazione Iterativa

Come seconda strategia di mitigazione, i modelli regolarizzati sono stati addestrati utilizzando un approccio di imputazione multivariata iterativa (**Iterative Imputer**). A differenza del KNN, che basa la ricostruzione sulla prossimità spaziale, l'Iterative Imputer modella ciascuna feature mancante come funzione delle altre variabili presenti nel dataset, ripetendo il processo in modo sequenziale (`max_iter=10`) per affinare le stime.

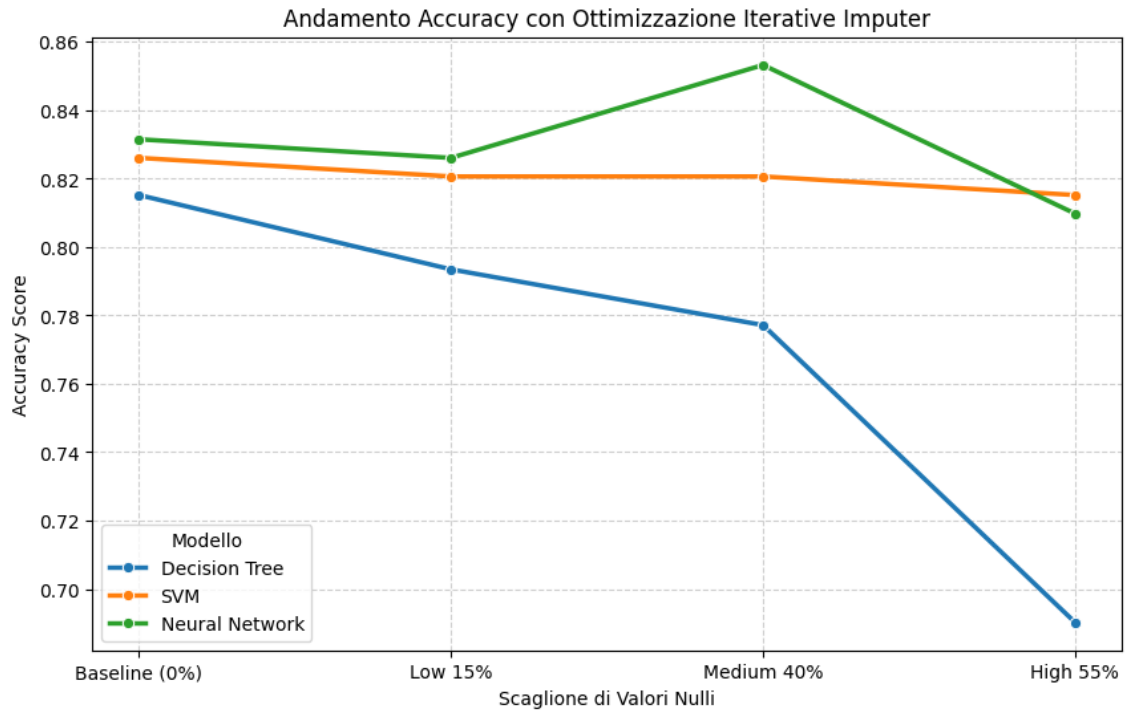


Figura 37: Andamento dell'Accuracy con ottimizzazione Iterative Imputer e Regolarizzazione.

| Livello              | Modello        | Accuracy | Precision | Recall | F1-Score |
|----------------------|----------------|----------|-----------|--------|----------|
| <i>Baseline (0%)</i> | Decision Tree  | 0.815    | 0.815     | 0.863  | 0.838    |
|                      | SVM            | 0.826    | 0.792     | 0.931  | 0.856    |
|                      | Neural Network | 0.832    | 0.845     | 0.853  | 0.849    |
| <i>Low 15%</i>       | Decision Tree  | 0.793    | 0.881     | 0.725  | 0.796    |
|                      | SVM            | 0.821    | 0.790     | 0.922  | 0.851    |
|                      | Neural Network | 0.821    | 0.822     | 0.863  | 0.842    |
| <i>Medium 40%</i>    | Decision Tree  | 0.777    | 0.828     | 0.755  | 0.790    |
|                      | SVM            | 0.821    | 0.785     | 0.931  | 0.852    |
|                      | Neural Network | 0.826    | 0.830     | 0.863  | 0.846    |
| <i>High 55%</i>      | Decision Tree  | 0.690    | 0.817     | 0.569  | 0.671    |
|                      | SVM            | 0.815    | 0.779     | 0.931  | 0.848    |
|                      | Neural Network | 0.804    | 0.844     | 0.794  | 0.818    |

Tabella 18: Metriche di performance per i modelli ottimizzati con Iterative Imputer.

- L'Iterative Imputer preserva in modo eccellente la separabilità dei dati richiesta dalla SVM. L'Accuracy rimane pressoché costante (tra 0.826 e 0.815) a ogni livello di degradazione. Ancora più notevole è la stabilità della *Recall* (0.931 al 55% di rumore), a conferma che queste stime permettono al kernel RBF di classificare correttamente quasi tutti i casi positivi (Figura 41).
- La Rete Neurale mostra ottime performance fino al 40% di rumore (0.826), per poi calare a 0.804 al 55%. A livelli così elevati di corruzione, il rumore generato dall'Iterative Imputer risulta più complesso da gestire per la rete rispetto alle medie locali del KNN, impedendone la corretta estrazione delle feature.

- Il Decision Tree è il modello più penalizzato: al 55 % di dati mancanti l'Accuracy scende a 0.690 e la *Recall* scende a 0.569. Questo dimostra che la complessa ricostruzione iterativa disturba l'algoritmo, rendendo le sue regole di partizionamento inefficaci per classificare i veri positivi.

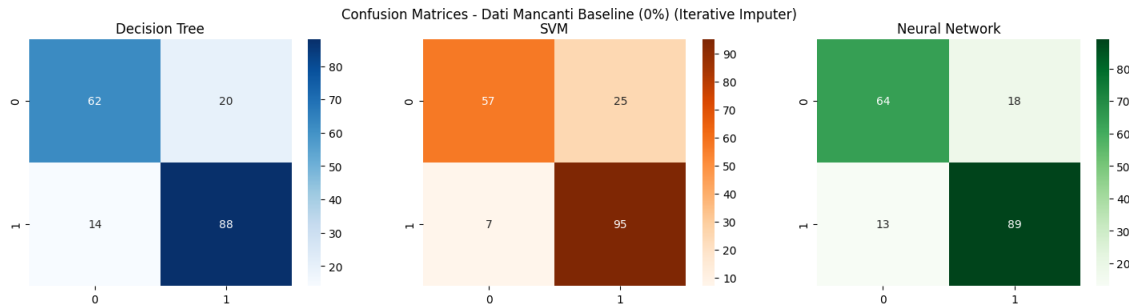


Figura 38: Matrici di confusione con Iterative Imputer e algoritmi regolarizzati (Baseline)

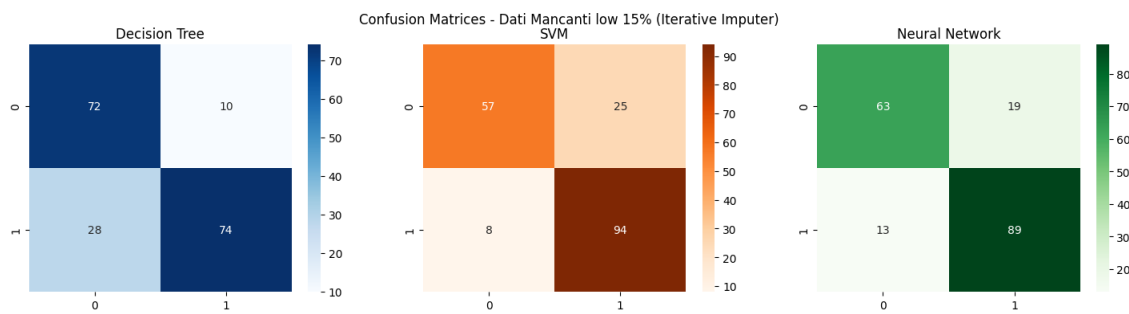


Figura 39: Matrici di confusione con Iterative Imputer e algoritmi regolarizzati (15% di valori nulli)

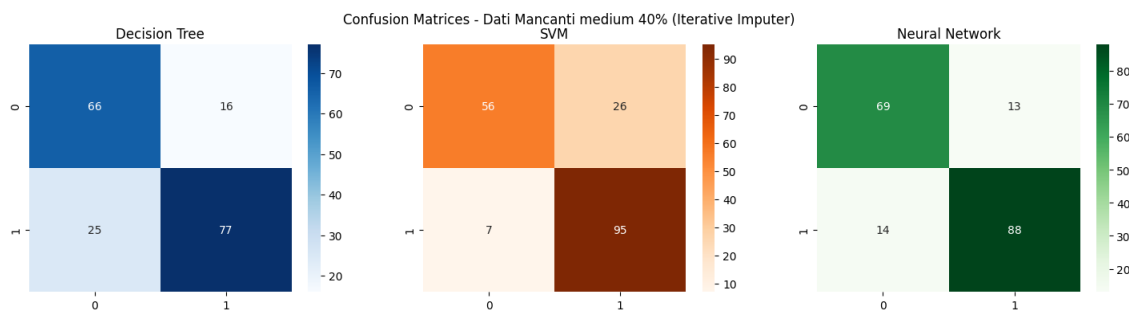


Figura 40: Matrici di confusione con Iterative Imputer e algoritmi regolarizzati (40% di valori nulli)



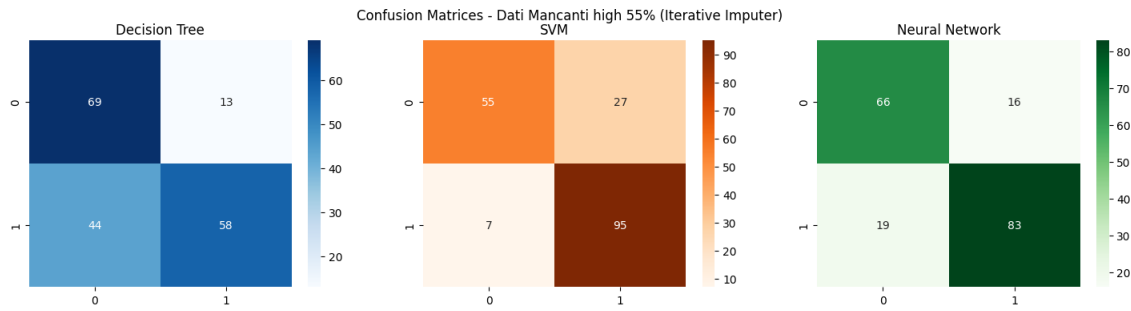


Figura 41: Matrici di confusione con Iterative Imputer e algoritmi regolarizzati (55% di valori nulli)

Per validare la reale capacità di generalizzazione dei modelli addestrati sui dati ricostruiti sequenzialmente, è stata applicata anche in questo caso una K-Fold Cross-Validation (K=5).

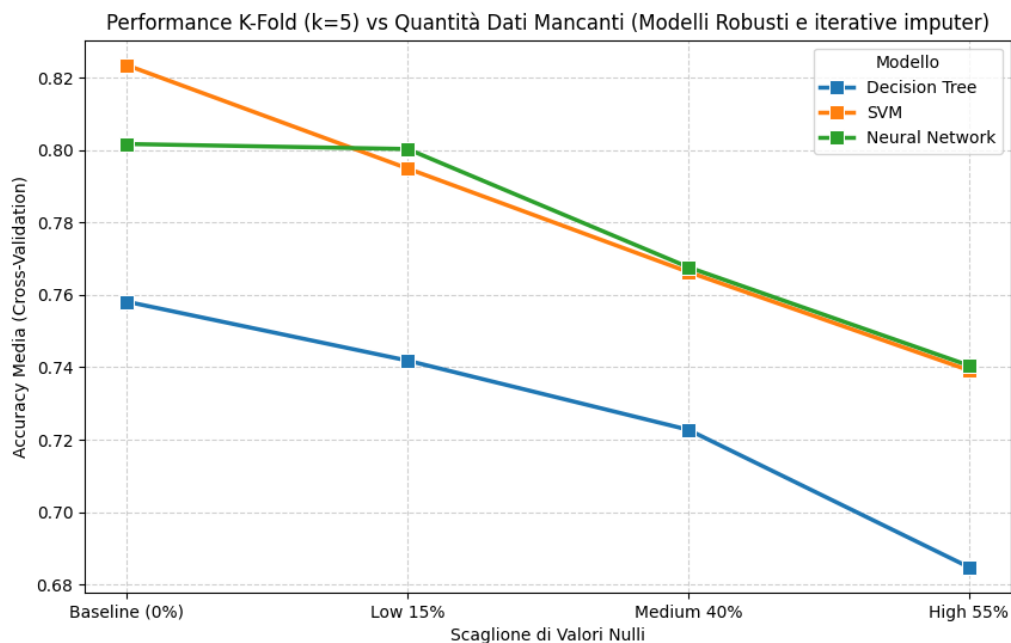


Figura 42: Andamento dell'Accuracy media in Cross-Validation (K=5) post-mitigazione con Iterative Imputer.

| Livello Dati Mancanti | Decision Tree | SVM   | Neural Network |
|-----------------------|---------------|-------|----------------|
| Baseline (0%)         | 0.758         | 0.823 | 0.800          |
| Low 15%               | 0.742         | 0.795 | 0.808          |
| Medium 40%            | 0.723         | 0.766 | 0.773          |
| High 55%              | 0.685         | 0.739 | 0.745          |

Tabella 19: Accuracy media in K-Fold CV (K=5) per i modelli ottimizzati con Iterative Imputer.

L'analisi dell'Accuracy media calcolata sui cinque fold (Tabella 19 e Figura 42) conferma il prevedibile deterioramento delle metriche all'aumentare del rumore, ma mo-

stra un quadro prestazionale decisamente superiore rispetto all'imputazione spaziale (KNN). In particolare:

- L'imputazione iterativa si conferma la tecnica più robusta per la generalizzazione. Sebbene il KNN sembrasse superiore nel singolo test estremo, la validazione incrociata dimostra che l'Iterative Imputer garantisce una stabilità nettamente maggiore, permettendo ai modelli di individuare pattern validi in modo costante.
- La Rete Neurale e la SVM mostrano un degrado molto contenuto: nello scenario estremo (55 %), mantengono metriche eccellenti (0.745 e 0.739), superando il KNN di oltre due punti percentuali. Ciò conferma che l'approccio iterativo preserva il segnale clinico nettamente meglio della semplice media spaziale.
- Pur confermandosi il modello meno stabile, anche il Decision Tree trae un chiaro vantaggio da questa tecnica. Al 55% di valori nulli raggiunge un'Accuracy di 0.685 (contro lo 0.658 del KNN), poiché la ricostruzione multivariata dei dati gli permette di generare regole di classificazione molto più coerenti

Per sintetizzare visivamente l'impatto delle diverse tecniche di mitigazione, la Figura 43 mostra il confronto diretto dell'Accuracy in Cross-Validation tra l'approccio di base (SimpleImputer) e le due strategie robuste (KNN e Iterative Imputer).

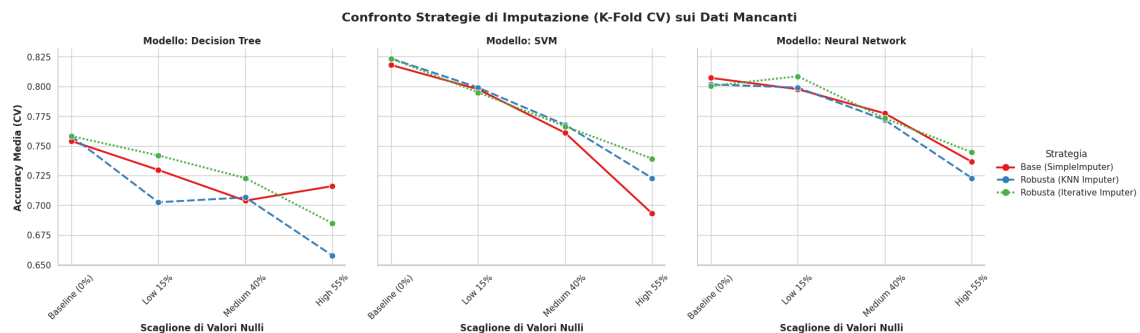


Figura 43: Confronto delle performance in K-Fold CV tra le diverse strategie di imputazione al variare dei dati mancanti.

Dall'osservazione dei tre scenari emergono dinamiche ben distinte:

- Nel caso del Decision Tree, l'imputazione iterativa (linea verde) si rivela la scelta ottimale per mantenere un andamento stabile e controllato del degrado prestazionale. Analizzando l'imputer di base (linea rossa), si osserva un calo lineare fino al 40% di valori nulli, seguito da un'inaspettata e anomala risalita al 55%; questo comportamento evidenzia l'imprevedibilità e l'instabilità dell'approccio semplice su un modello gerarchico.
- Per la Support Vector Machine, il grafico illustra una chiara gerarchia d'efficacia: il SimpleImputer è la tecnica che si comporta peggio, subendo un crollo severo nello scenario estremo. L'applicazione del KNN Imputer ne migliora sensibilmente la tenuta, ma è con il modello iterativo che l'algoritmo ottiene la massima stabilità predittiva.

- Le Reti Neurali si confermano l'architettura più robusta in assoluto, mantenendosi sempre al di sopra della soglia dello 0.720 anche nello scenario più critico. Analizzando il dettaglio delle imputazioni, si nota che il KNN risulta essere la strategia meno performante, il Simple Imputer si posiziona su livelli intermedi, mentre l'Iterative Imputer garantisce anche in questo caso il risultato migliore in assoluto.

In conclusione, l'analisi dimostra che tutti e tre i modelli raggiungono le performance ottimali utilizzando l'Iterative Imputer. Questa tecnica si rivela in modo inequivocabile la strategia di imputazione migliore e più sicura da adottare quando si elaborano dataset caratterizzati da un alto tasso di valori nulli.

## 9.2 Mitigazione dei valori duplicati

A differenza delle altre tipologie di rumore, la presenza di duplicati non degrada in modo diretto le prestazioni misurate sul test set indipendente (Sezione precedente), bensì inquina la stima ottenuta tramite Cross-Validation, alterando la percezione della reale capacità di generalizzazione dei modelli. La mitigazione deve quindi agire a monte della procedura di validazione, sulla composizione stessa del training set. Sono state confrontate due strategie, applicate in combinazione con la medesima pipeline di modelli utilizzata nella Sezione precedente:

- **Rimozione dei duplicati:** identificazione ed eliminazione delle righe integralmente identiche (`drop_duplicates()`, applicato a feature e target congiuntamente), per evitare che il modello attribuisca peso eccessivo a osservazioni ripetute e, soprattutto, per impedire che copie della stessa riga finiscano simultaneamente nei fold di training e di validazione;
- **Aggregazione intelligente:** anziché rimuovere semplicemente le righe duplicate, le osservazioni identiche vengono raggruppate sulla base delle sole variabili predittive, assegnando come etichetta la *moda* del target tra le occorrenze duplicate. Questa tecnica è pensata per gestire anche i casi in cui i duplicati presentino etichette discordanti (rumore di etichettatura), scenario che non si verifica in questo esperimento poiché la duplicazione è stata simulata per copia esatta della riga, target incluso.

Applicando entrambe le tecniche, il training set torna, per tutti i livelli di rumore, all'esatta dimensione originale di 735 righe (Tabella 20).

| Livello       | Righe rimosse | Dimensione finale |
|---------------|---------------|-------------------|
| Baseline (0%) | 0             | 735               |
| Low (15%)     | 111           | 735               |
| Medium (40%)  | 295           | 735               |
| High (55%)    | 405           | 735               |

Tabella 20: Effetto della rimozione dei duplicati sulla dimensione del training set.

### 9.2.1 Rimozione dei duplicati

La prima strategia elimina, ad ogni fold della 5-Fold Cross-Validation, le righe integralmente duplicate prima dell'addestramento. Le matrici di confusione aggregate sui 5 fold sono riportate nelle Figure 44–47.

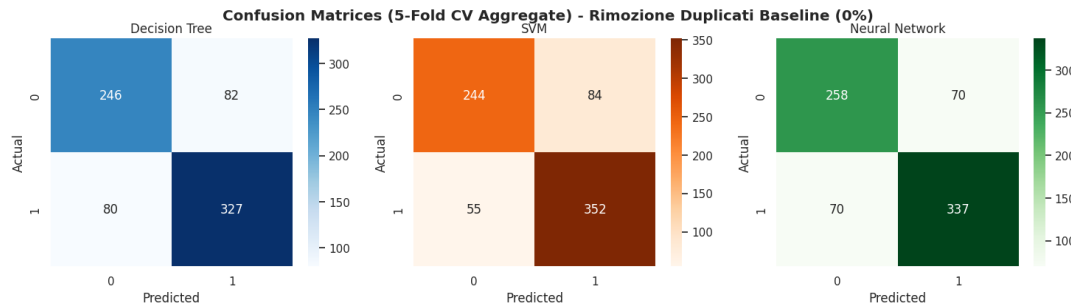


Figura 44: Matrici di confusione (CV aggregata) dopo rimozione dei duplicati – Baseline (0%)

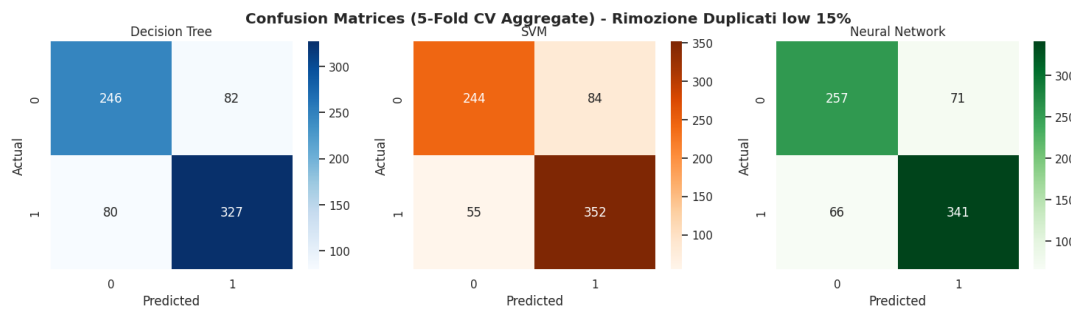


Figura 45: Matrici di confusione (CV aggregata) dopo rimozione dei duplicati – 15%

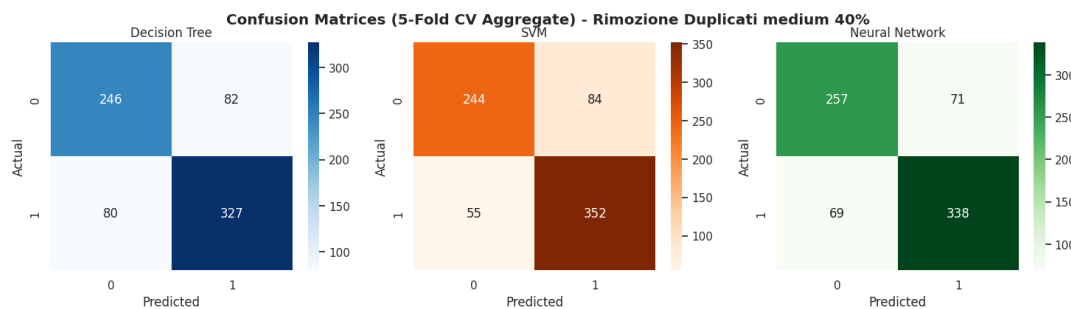


Figura 46: Matrici di confusione (CV aggregata) dopo rimozione dei duplicati – 40%

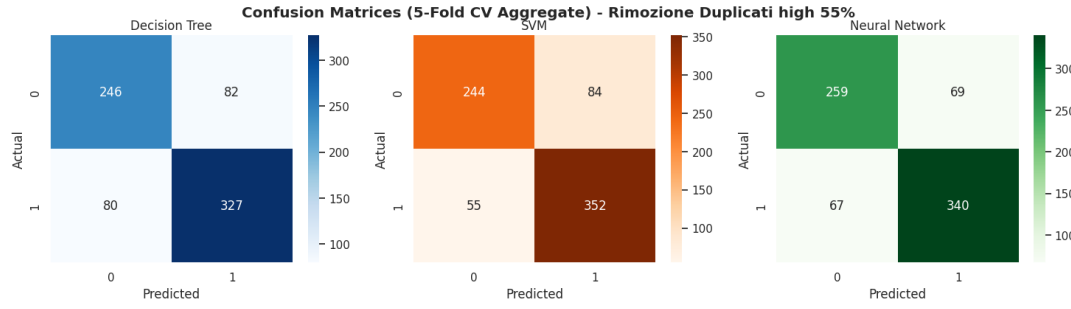


Figura 47: Matrici di confusione (CV aggregata) dopo rimozione dei duplicati – 55%

Poiché la rimozione delle righe identiche riporta il training set esattamente alla sua composizione originale a ogni livello di duplicazione iniziale, le matrici ottenute nei quattro scenari risultano pressoché indistinguibili: per Decision Tree e SVM, algoritmi deterministici a parità di `random_state`, i valori sono numericamente identici; la Rete Neurale mostra invece oscillazioni marginali, imputabili unicamente alla componente stocastica dell’inizializzazione dei pesi e non al livello di rumore. Questo conferma che il problema introdotto dai duplicati non è di natura predittiva, ma puramente procedurale: rimossa la ridondanza, ciascun modello si comporta esattamente come nello scenario privo di rumore.

### 9.2.2 Aggregazione intelligente

La seconda strategia raggruppa le osservazioni identiche sulla base delle sole variabili predittive e assegna come etichetta la moda del target tra le occorrenze duplicate. Le matrici di confusione ottenute sono riportate nelle Figure 48–51.

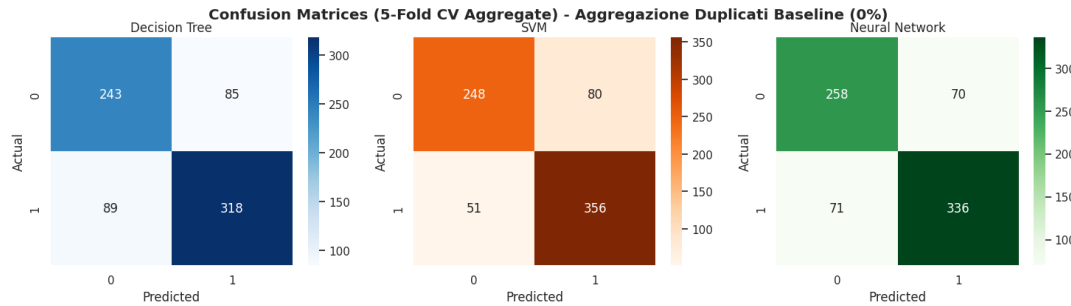


Figura 48: Matrici di confusione (CV aggregata) dopo aggregazione intelligente – Baseline (0%)

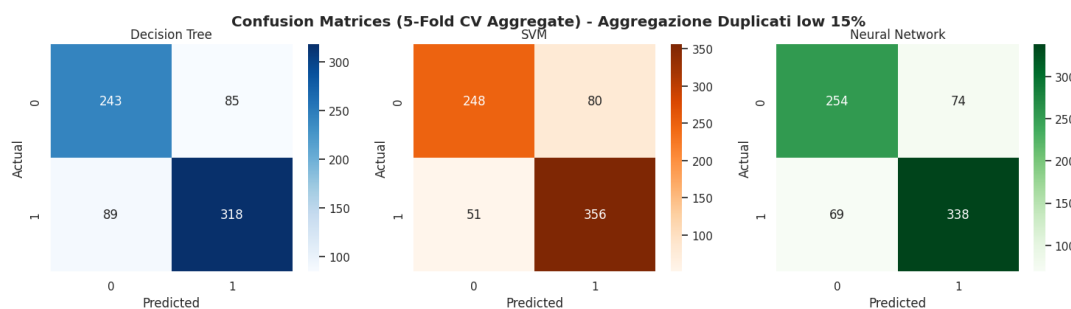


Figura 49: Matrici di confusione (CV aggregata) dopo aggregazione intelligente – 15%

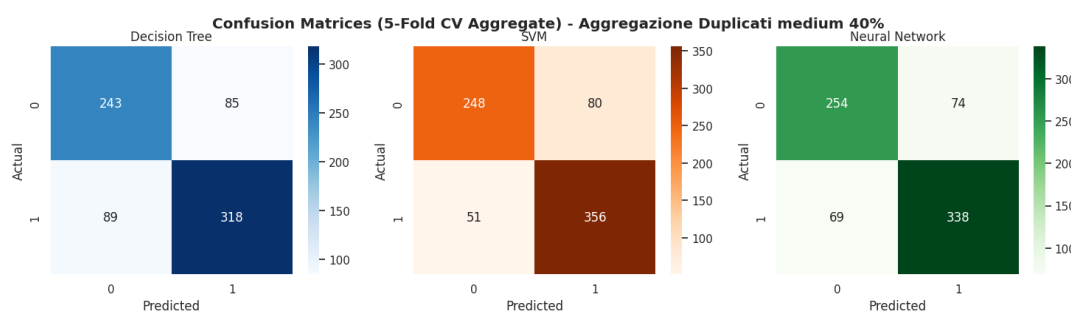


Figura 50: Matrici di confusione (CV aggregata) dopo aggregazione intelligente – 40%

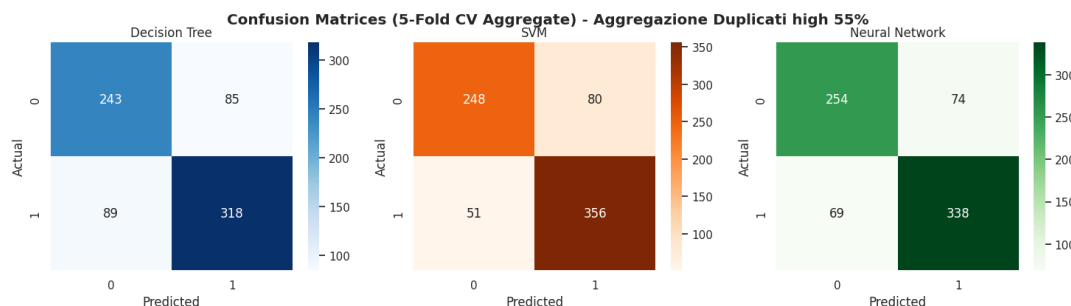


Figura 51: Matrici di confusione (CV aggregata) dopo aggregazione intelligente – 55%

Trattandosi in questo esperimento di una duplicazione per copia esatta, priva di rumore di etichettatura, l'aggregazione tramite moda recupera esattamente lo stesso sottoinsieme di righe uniche ottenuto con la semplice rimozione: le due strategie, pur concettualmente distinte, convergono verso risultati numericamente equivalenti. La Tabella 21 riporta l'Accuracy media di Cross-Validation ottenuta dopo la pulizia, valida indistintamente per entrambe le tecniche.

| Livello       | Decision Tree | SVM   | Neural Network |
|---------------|---------------|-------|----------------|
| Baseline (0%) | 0.763         | 0.822 | 0.811          |
| Low (15%)     | 0.763         | 0.822 | 0.797          |
| Medium (40%)  | 0.763         | 0.822 | 0.803          |
| High (55%)    | 0.763         | 0.822 | 0.801          |

Tabella 21: Accuracy media di CV (5-fold) dopo pulizia dei duplicati (rimozione/aggregazione, dati equivalenti).

L'analisi della tabella conferma quanto osservato graficamente:

- Il Decision Tree e la SVM mostrano un'Accuracy media perfettamente costante nei quattro scenari (rispettivamente 0.763 e 0.822): una volta eliminata la ridondanza, l'algoritmo lavora sempre sullo stesso training set, indipendentemente dal livello di duplicazione simulato in partenza;
- La Rete Neurale presenta un'oscillazione contenuta tra 0.797 e 0.811, interamente attribuibile alla natura stocastica dell'addestramento (inizializzazione dei pesi non vincolata da un seed) e non a un reale effetto residuo del rumore;
- Rispetto ai valori artificialmente elevati restituiti dalla Cross-Validation eseguita sui dati grezzi (Tabella 13), entrambe le strategie di pulizia restituiscono stime più basse ma affidabili, coerenti con l'Accuracy osservata sul test set indipendente (Tabella 12).

### 9.2.3 Confronto tra le strategie e conclusioni

Il grafico seguente confronta l'accuratezza media ottenuta tramite Cross-Validation sui dati grezzi con duplicati rispetto a quella ottenuta applicando le due strategie di pulizia.

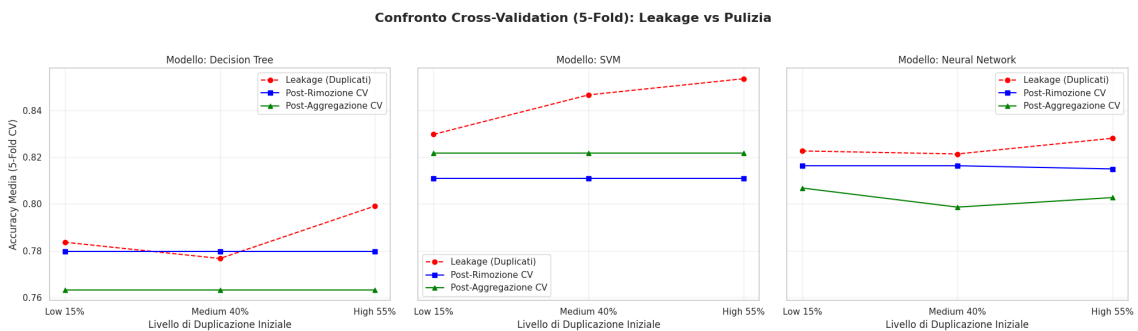


Figura 52: Confronto tra le strategie di gestione dei duplicati in Cross-Validation per i tre modelli analizzati.

Come si evince dalla Figura 52, la valutazione sui dati non trattati (linea tratteggiata rossa) soffre di un evidente fenomeno di *data leakage*: all'aumentare della percentuale di duplicati, l'accuratezza di Cross-Validation cresce fittiziamente, in modo particolarmente marcato per i modelli più complessi come SVM e Rete Neurale, poiché le stesse osservazioni finiscono per essere presenti simultaneamente nei

fold di addestramento e di validazione. Al contrario, le curve post-rimozione e post-aggregazione risultano perfettamente piatte e sovrapposte al variare del livello di duplicazione iniziale, poiché entrambe le tecniche riconducono il training set all'esatta composizione della Baseline originale.

In conclusione, a differenza dei valori mancanti e dei dati sporchi, il rumore da duplicazione non richiede tecniche di regolarizzazione del modello, ma esclusivamente un intervento di pulizia a monte della procedura di validazione: la scelta tra rimozione semplice e aggregazione intelligente è quindi ininfluente in assenza di etichette discordanti, e dovrebbe orientarsi verso l'aggregazione intelligente solo negli scenari reali in cui i duplicati possano portare informazioni di etichetta contrastanti.

### 9.3 Mitigazione dei dati sporchi

Per contrastare l'effetto dei dati "sporchi" (rumore gaussiano e swap di valori binari), che si sono rivelati la tipologia di rumore più impattante, è stata adottata una pipeline di preprocessing robusto composta da:

- **RobustScaler**: normalizzazione basata su mediana e range interquartile, meno sensibile agli outlier rispetto allo **StandardScaler** classico;
- **PCA (Principal Component Analysis)**, mantenendo il numero minimo di componenti necessario a spiegare il **95%** della varianza totale, con l'obiettivo di filtrare parte del rumore concentrato nelle componenti a bassa varianza;
- modelli regolarizzati (Decision Tree con `max_depth=4`, `min_samples_split=20`, `min_samples_leaf=15`; SVM con  $C = 0.1$ ).

Il numero di componenti principali necessarie per raggiungere il 95% della varianza spiegata, per ciascun livello di rumore, è riportato in Tabella 22.

| Livello       | Componenti principali (95% varianza) |
|---------------|--------------------------------------|
| Baseline (0%) | 14                                   |
| Low (15%)     | 16                                   |
| Medium (40%)  | 14                                   |
| High (55%)    | 16                                   |

Tabella 22: Numero di componenti principali necessarie per spiegare il 95% della varianza, per livello di rumore.



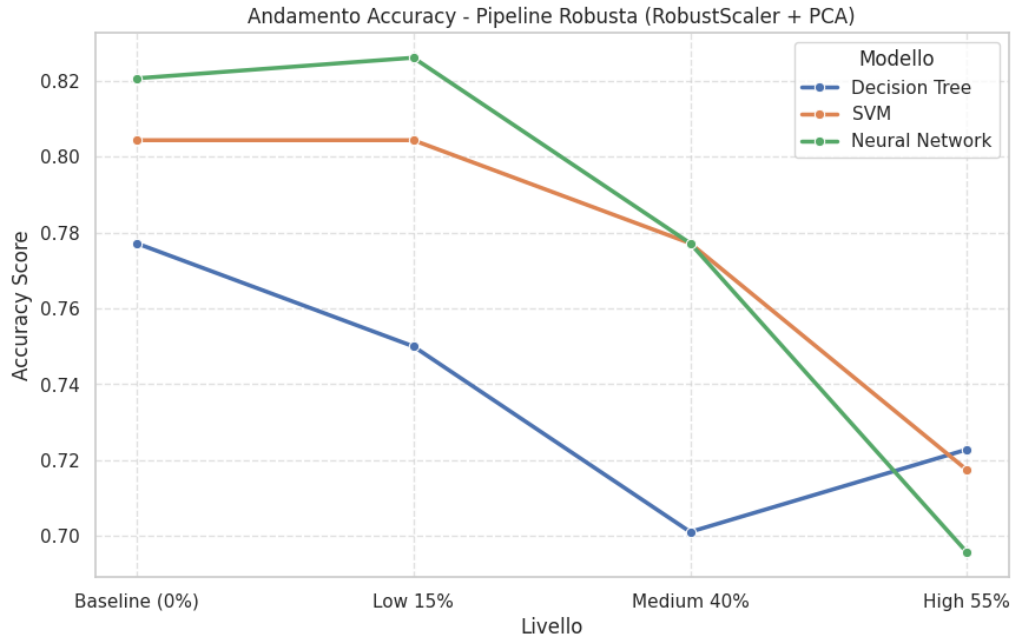


Figura 53: Andamento dell'Accuracy con pipeline robusta applicata al rumore gaussiano e di swap.

| Livello Rumore       | Modello        | Accuracy | Precision | Recall | F1-Score |
|----------------------|----------------|----------|-----------|--------|----------|
| <i>Baseline (0%)</i> | Decision Tree  | 0.777    | 0.748     | 0.902  | 0.818    |
|                      | SVM            | 0.804    | 0.784     | 0.892  | 0.835    |
|                      | Neural Network | 0.821    | 0.835     | 0.843  | 0.839    |
| <i>Low 15%</i>       | Decision Tree  | 0.750    | 0.780     | 0.765  | 0.772    |
|                      | SVM            | 0.804    | 0.784     | 0.892  | 0.835    |
|                      | Neural Network | 0.826    | 0.824     | 0.873  | 0.848    |
| <i>Medium 40%</i>    | Decision Tree  | 0.701    | 0.747     | 0.696  | 0.721    |
|                      | SVM            | 0.777    | 0.775     | 0.843  | 0.808    |
|                      | Neural Network | 0.777    | 0.802     | 0.794  | 0.798    |
| <i>High 55%</i>      | Decision Tree  | 0.723    | 0.787     | 0.686  | 0.733    |
|                      | SVM            | 0.717    | 0.736     | 0.765  | 0.750    |
|                      | Neural Network | 0.696    | 0.735     | 0.706  | 0.720    |

Tabella 23: Metriche di performance (test set) per i modelli addestrati con pipeline robusta (RobustScaler + PCA) sui dati con rumore attivo.

Le metriche (Tabella 23 e Figura 53) confermano che il rumore attivo è molto più distruttivo della semplice assenza di dati. Pur applicando una robusta pipeline di filtraggio, il degrado prestazionale è marcato:

- La Support Vector Machine resta la più resiliente fino a livelli intermedi di rumore (0.804 al 15% e 0.777 al 40%). Tuttavia, al 55% di corruzione, la distorsione geometrica supera le capacità di filtraggio della PCA: l'Accuracy crolla a 0.717 e la *Recall* scende drasticamente a 0.765.
- La Rete Neurale mostra l'andamento più drammatico. Dopo un picco iniziale al 15% (0.826), subisce un collasso verticale, chiudendo come peggior modello

al 55% (0.696). Con oltre metà dei dati contraffatti, la rete finisce per assimilare il rumore come pattern valido, disorientando i pesi sinaptici in fase di addestramento.

- Il Decision Tree conferma la sua instabilità. Il finto "rimbalzo" dell'Accuracy al 55% (0.723) è smentito dalla *Recall*, che crolla a 0.686 (Figura 57). Di fatto, il modello sta perdendo quasi un terzo dei veri positivi, rendendo le sue regole di split totalmente inaffidabili.

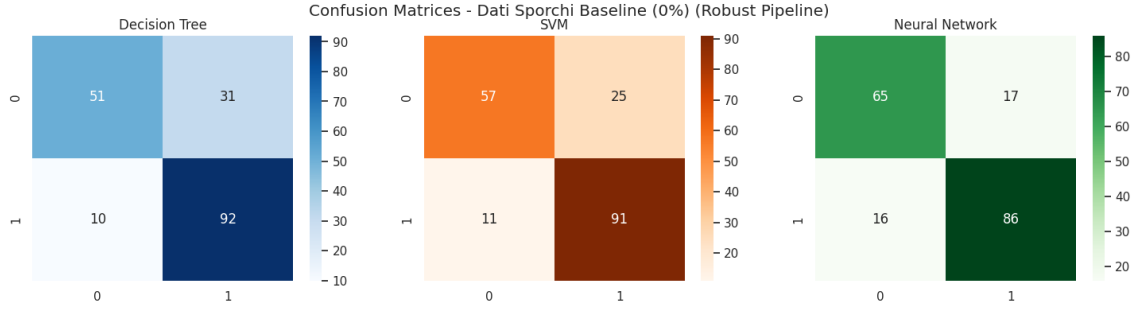


Figura 54: Matrici di confusione dei modelli con pipeline robusta sui dati originali (Baseline 0%).

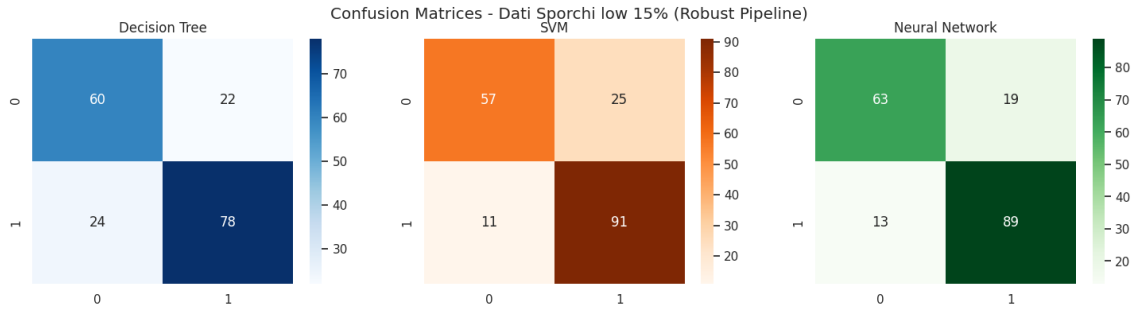


Figura 55: Matrici di confusione dei modelli con pipeline robusta al 15% di rumore (gaussiano e swap).

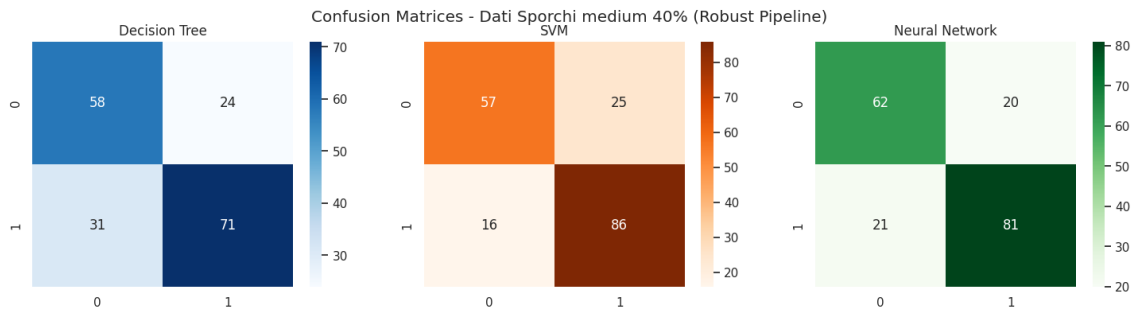


Figura 56: Matrici di confusione dei modelli con pipeline robusta al 40% di rumore (gaussiano e swap).

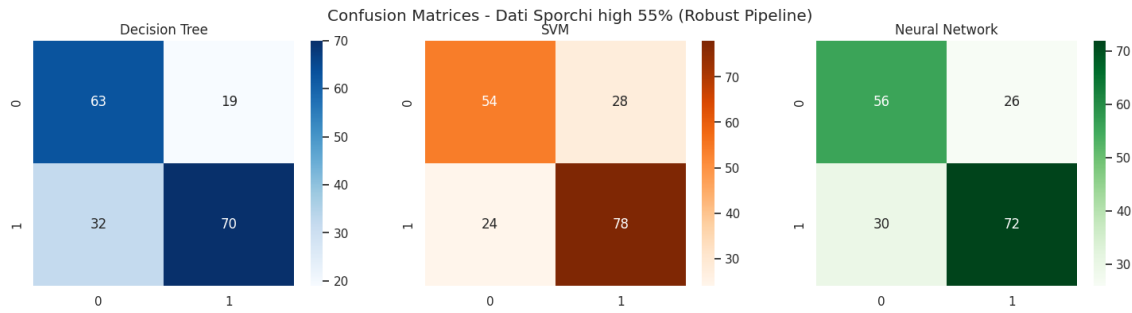


Figura 57: Matrici di confusione dei modelli con pipeline robusta al 55% di rumore (gaussiano e swap). Si noti l'elevato numero di falsi negativi nel Decision Tree.

Per ottenere una stima più rigorosa della reale capacità di generalizzazione della pipeline robusta, i modelli sono stati sottoposti a una K-Fold Cross-Validation (K=5).

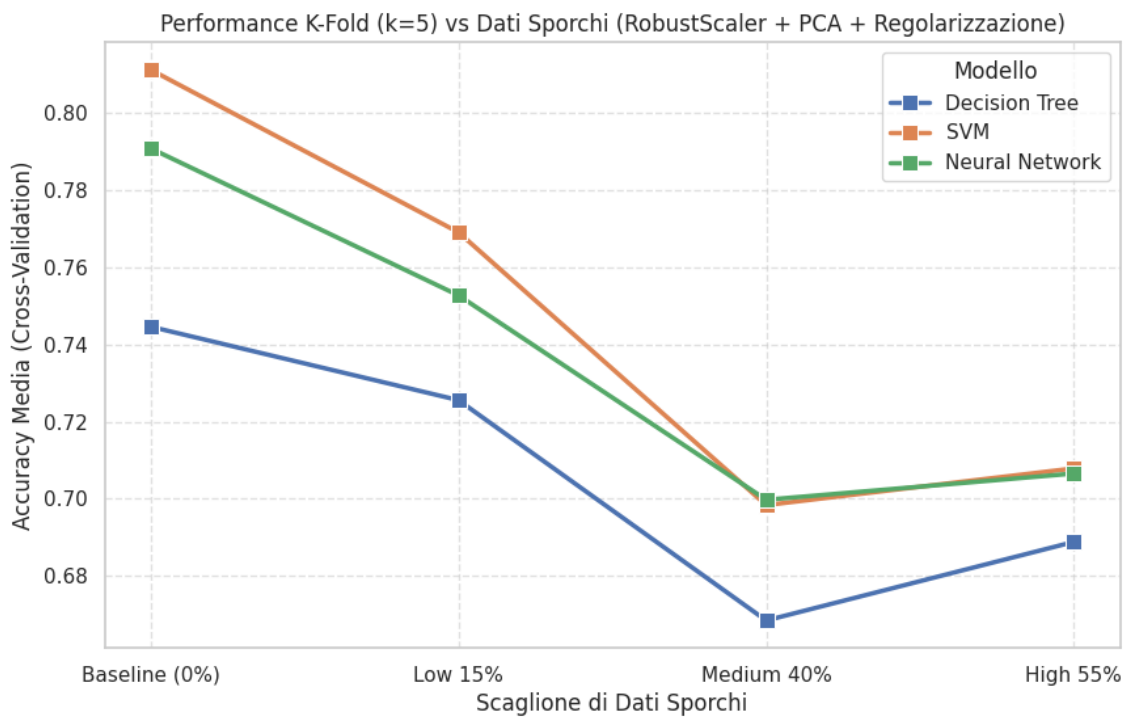


Figura 58: Andamento dell'Accuracy media in Cross-Validation (K=5) per i modelli addestrati con pipeline robusta al variare del rumore gaussiano e di swap.

| Livello Rumore | Decision Tree | SVM   | Neural Network |
|----------------|---------------|-------|----------------|
| Baseline (0%)  | 0.745         | 0.811 | 0.791          |
| Low 15%        | 0.726         | 0.769 | 0.753          |
| Medium 40%     | 0.668         | 0.698 | 0.700          |
| High 55%       | 0.689         | 0.708 | 0.707          |

Tabella 24: Accuracy media in K-Fold CV (K=5) per la pipeline robusta applicata ai dati sporchi.

I risultati medi sui cinque fold (Tabella 24 e Figura 58) ridefiniscono le dinamiche osservate nel test singolo e mostrano un comportamento predittivo particolare legato all'intensità del rumore:

- Il punto di minima performance assoluta si registra per tutte le architetture al livello di corruzione *Medium* (40%). In questa fascia, il mix tra rumore gaussiano e swap parziale delle etichette genera un'elevata entropia nel dataset, disturbando le capacità di estrazione delle feature.
- Passando allo scenario estremo *High* (55%), le reazioni dei modelli si diversificano. La Support Vector Machine e la Rete Neurale registrano una risalita puramente marginale, stabilizzandosi di fatto in area 0.70. Invece, il Decision Tree evidenzia un risalita prestazionale più marcata, risalendo da 0.668 a 0.689.
- Il falso recupero del Decision Tree mostra la vulnerabilità strutturale: nonostante la forte potatura, i suoi split rigidi seguono il "segnale invertito". Superato il 50% di corruzione, la classe errata diventa maggioritaria e l'albero si limita a invertire le regole dei nodi. Al contrario, SVM e Rete Neurale, dotate di confini decisionali continui, riescono a mantenere ancora una certa resistenza.

In conclusione, l'immissione di dati sporchi si conferma la criticità più severa per l'apprendimento automatizzato. Nonostante la mitigazione utilizzata (PCA e regolarizzazione), l'analisi delinea una chiara gerarchia architetturale: la Support Vector Machine si afferma come l'algoritmo più resiliente e affidabile, seguita dalla Rete Neurale, mentre il Decision Tree risulta strutturalmente inadeguato a operare in contesti ad elevata corruzione informativa.

## 10 Confronto finale tra le strategie

Per fornire una visione d'insieme, i risultati di tutte le strategie di gestione del rumore sono stati raccolti in un unico confronto grafico (Figura 59), organizzato per tipologia di rumore e per modello.

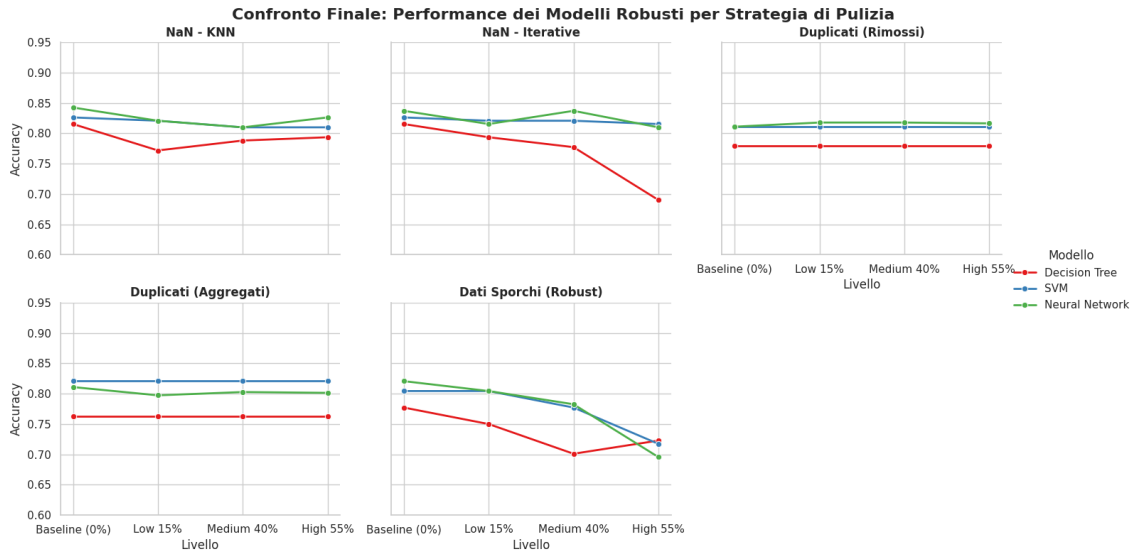


Figura 59: Confronto finale delle prestazioni dei modelli robusti per strategia di pulizia dei dati

Dall'osservazione visiva delle curve di degradazione emergono tre dinamiche chiave:

- La Support Vector Machine domina i grafici, confermando visivamente la sua superiore stabilità trasversale rispetto agli altri modelli.
- L'assenza di dati (NaN) e i duplicati presentano curve post-mitigazione piatte e controllate, a dimostrazione che le tecniche di imputazione (KNN/Iterative) e di pulizia neutralizzano quasi totalmente il problema.
- Il riquadro dei Dati Sporchi è l'unico a mostrare un crollo ineliminabile delle curve (nello scenario *High*), confermandosi visivamente come l'ostacolo più arduo da filtrare persino per la pipeline robusta.

## 11 Conclusioni

Il progetto ha permesso di analizzare in modo sistematico l'impatto di tre diverse tipologie di corruzione dei dati (valori mancanti, duplicati, dati sporchi) sulle prestazioni di tre architetture di Machine Learning (Decision Tree, SVM, Rete Neurale) in un dominio diagnostico.

I risultati dimostrano che:

- l'impatto del rumore è fortemente eterogeneo: la semplice assenza di dati (NaN) è arginabile algoritmicamente, l'alterazione dei valori (dati sporchi) distrugge irreversibilmente il segnale informativo, mentre i duplicati agiscono in modo ingannevole causando *data leakage* e sovrastimando le performance se non rimossi prima della Cross-Validation;
- l'impiego di tecniche di regolarizzazione (come il pruning, l'impostazione di un *soft margin*, il dropout e l'early stopping) risulta cruciale e determinante per impedire ai modelli di andare in overfitting sul rumore artificiale;

- non esiste un approccio di preprocessing universale, in quanto la mitigazione deve essere rigorosamente mirata alla specifica anomalia: ciò che salva un modello dai valori mancanti (come le tecniche di imputazione) non ha infatti alcuna utilità contro lo swap delle etichette.

Come possibile sviluppo futuro, si potrebbero esplorare tecniche di rilevamento automatico delle anomalie per selezionare in modo adattivo la strategia di preprocessing, oltre a estendere l'analisi a modelli *ensemble* (es. Random Forest o Gradient Boosting) per verificarne la generalizzabilità.

In conclusione, il lavoro dimostra che la qualità del dato e la sua corretta pulizia incidono sul successo predittivo in misura pari, se non superiore, alla scelta dell'algoritmo stesso.